MULTIVIEW: A Portable C99 System for Fixed-Camera Scene Analysis and 3D Model Extraction

A self-contained overview of the system and its mathematics

Project documentation — v0.6.0

August 4, 2026

Abstract

MULTIVIEW is a small, dependency-free C99 library for metric scene analysis and 3D model extraction from the synchronized video streams of *two stationary cameras* observing an overlapping volume, optionally augmented by a collection of still photographs and, in a later stage, range-finder measurements. Calibration is part of the system: the rig is calibrated from views of a printed letter-page checkerboard of known square size, or from a computer display under our control showing synchronized Gray-code patterns. This document doubles as a self-study overview of the underlying mathematics: each concept is developed from first principles, cross-linked to the modules that implement it, illustrated by a *hand-computable* worked example, and sourced in the bibliography. All quantitative claims (section 13) are reproducible from the deterministic test programs in the repository.

Contents

1 Introduction and capabilities

The target deployment is deliberately narrow and fully under our control: *fixed* cameras — two in the reference rig — are rigidly mounted and observe the same working volume. "Fixed" is the load-bearing word: the contrast is with mobile-camera systems (handheld, vehicle- or drone-mounted), which must solve ego-motion before they can measure anything. Here the cameras never move, so every rig-dependent quantity is computed once — and a *mobile* camera entering the volume is not a rival architecture but just another tracked mover, one that happens to carry a sensor (subsection 11.1). The system first *calibrates itself* from views of a known planar target (section 5); thereafter video is processed frame by frame. Because the cameras do not move, every geometric quantity that depends only on the rig — projection matrices, the fundamental matrix, the rectifying homographies — is computed *once* and reused for every frame, which is the central efficiency win of the stationary-rig assumption.

The implemented core (v0.4.0) provides:

1. plane-based calibration of intrinsics, distortion, and pose from a printed letter-page checkerboard or from a controlled display showing Gray-code patterns (section 5);
2. a calibrated pinhole camera model with Brown radial–tangential lens distortion, forward projection and ray back-projection (section 4);
3. exact two-view epipolar geometry from the calibration, and the normalized 8-point algorithm to estimate or verify it from image correspondences (section 6);
4. N-view linear triangulation of corresponding image points into metric 3D, with reprojection-error reporting (section 7);
5. planar stereo rectification of the pair, and dense correspondence by block matching with sub-pixel refinement, giving per-pixel depth (section 8);

6. the calibration-subsystem front half: the spec-v1 composite screen generator with its shipped M-array, a synthetic renderer, and the blind pattern reader — corners, identities, orientation, and frame counter from a single frame (subsection 5.9);

7. TSDF fusion over enriched samples with marching-tetrahedra mesh extraction — the first model representation beyond point clouds (subsection 9.2);

8. point-cloud assembly and PLY export[1] for inspection in standard mesh tools, plus minimal PGM image I/O (section 9).

Planned extensions — structure-from-motion over an auxiliary photo collection and fusion of range-finder data — are designed but not yet implemented; their theory and integration points are described in section 11 so that the code can grow into them without rearchitecting.

How to read this document. section 3 collects the small amount of linear algebra everything else stands on. section 4–section 8 then follow the data through the system: camera model, calibration, two-view geometry, triangulation, dense stereo. Every section ends in a worked example with numbers chosen so the computation fits on paper; doing them by hand *is* the intended self-study exercise, and each can then be checked against the library.

Design constraints. The library is strict C99 with no dependencies beyond `libm`, row-major `double` arrays throughout, and fully deterministic (no clocks, no ambient randomness; the demos use fixed-seed generators). This keeps it portable from desktop to embedded targets, and keeps every experiment repeatable — a prerequisite for the controlled-environment, learning-oriented use the project is intended for.

2 System overview

Processing has a one-time phase and a per-frame phase:

$$\underbrace{\text{target views} \to \text{calibrate}}_{\text{once, } \texttt{target/graycode/calib}} \qquad \underbrace{\text{2 frames} \to \text{undistort} \to \text{rectify} \to \text{match} \to \text{depth} \to \text{cloud}}_{\text{per frame, } \texttt{cam/rectify/stereo/triangulate/cloud}}$$

Sparse operation (feature correspondences instead of dense matching) uses the same chain with `epipolar` providing the correspondence-validation gate. Because the rig is static, temporal accumulation over video frames is a simple union of per-frame clouds in the fixed world frame; moving objects appear as temporally inconsistent points and can be segmented by comparing frames — the entry point for scene *analysis* beyond reconstruction.

3 Mathematical toolkit

Four ideas carry the whole system; everything later is an application.

[1] PLY, the *polygon file format* (also "Stanford triangle format"), is the de-facto interchange format for point clouds and meshes. Its *ASCII* variant is plain text: a short human-readable header declaring the elements and their properties, then one line per vertex — trivially writable from C99, diffable, and loadable by every common mesh tool. See section 9 for a complete example file.

Module	Header	Responsibility
`mat`	`mv/mat.h`	small dense linear algebra, Jacobi SVD, null vectors
`cam`	`mv/cam.h`	camera model, projection, rays, distortion
`target`	`mv/target.h`	printed checkerboard model + printable rendering
`graycode`	`mv/graycode.h`	display-based active target (Gray-code patterns)
`calib`	`mv/calib.h`	homographies; Zhang calibration; radial estimate
`pattern`	`mv/pattern.h`	spec-v1 calibration screen, shipped M-array
`render`	`mv/render.h`	synthetic camera views of the display plane
`reader`	`mv/reader.h`	blind pattern decoding: corners, ids, counter
`epipolar`	`mv/epipolar.h`	$\mathbf{F}$ from calibration; normalized 8-point; $\mathbf{E}$
`triangulate`	`mv/triangulate.h`	N-view DLT triangulation, reprojection RMS
`rectify`	`mv/rectify.h`	Fusiello rectification, disparity→depth
`stereo`	`mv/stereo.h`	dense block matching (SAD, sub-pixel)
`img`	`mv/img.h`	PGM (P5) image I/O
`cloud`	`mv/cloud.h`	point cloud container, PLY export
`tsdf`	`mv/tsdf.h`	TSDF fusion over enriched samples; meshing

Table 1: Module layout. `mv/mv.h` is the umbrella header.

3.1 Homogeneous coordinates

A 2D point (u, v) is represented as any triple (wu, wv, w), $w \neq 0$; a 3D point (X, Y, Z) as $(X, Y, Z, 1)$ up to scale. The payoff: projection, homographies and rigid motions all become *linear* maps, and points at infinity ($w = 0$) need no special case. We write $\simeq$ for equality up to a nonzero scale.

Example 1 (dehomogenization)**.** (120, 140, 2) $\simeq$ (60, 70, 1): both name the pixel $(60, 70)$. Conversely the direction "straight down the u axis" is $(1, 0, 0)$, which no finite pixel represents.

3.2 Rotations

A rotation matrix $\mathbf{R} \in SO(3)$ satisfies $\mathbf{R}^\top\mathbf{R} = \mathbf{I}$, $\det \mathbf{R} = 1$; its rows are the axes of the new frame expressed in the old. Consequently $\mathbf{R}^{-1} = \mathbf{R}^\top$ — inverting a rigid transform never requires a matrix inversion.

Example 2 (rotation about y)**.** $\mathbf{R}_y(90°) = \left(\begin{smallmatrix} 0 & 0 & -1 \\ 0 & 1 & 0 \\ 1 & 0 & 0 \end{smallmatrix}\right)$ sends the world z axis $(0, 0, 1)$ to $(-1, 0, 0)$: a camera yawed 90° sees the world's $-x$ direction ahead. Check $\mathbf{R}^\top\mathbf{R} = \mathbf{I}$ by inspection: the rows are orthonormal.

3.3 The cross-product matrix

For $\mathbf{v} = (v_1, v_2, v_3)$,

$$[\mathbf{v}]_\times = \begin{pmatrix} 0 & -v_3 & v_2 \\ v_3 & 0 & -v_1 \\ -v_2 & v_1 & 0 \end{pmatrix}, \qquad [\mathbf{v}]_\times\mathbf{w} = \mathbf{v} \times \mathbf{w}. \tag{1}$$

It is rank 2 with null vector $\mathbf{v}$ itself — the algebraic reason epipoles exist (section 6).

Example 3 (skew matrix)**.** $\mathbf{v} = (1, 0, 0)$: $[\mathbf{v}]_\times = \left(\begin{smallmatrix} 0 & 0 & 0 \\ 0 & 0 & -1 \\ 0 & 1 & 0 \end{smallmatrix}\right)$, and $[\mathbf{v}]_\times(0, 1, 0)^\top = (0, 0, 1)^\top = \mathbf{v}\times\mathbf{w}$.

3.4 Homogeneous least squares and the SVD

Every estimation problem in this system reduces to: given $\mathbf{A}$ $(m \times n)$, find $\mathbf{x} \neq 0$ minimizing $\|\mathbf{Ax}\|$ subject to $\|\mathbf{x}\| = 1$. The minimizer is the right singular vector of the smallest singular value of $\mathbf{A} = \mathbf{U}\,\mathrm{diag}(\sigma_1 \geq \cdots \geq \sigma_n)\,\mathbf{V}^\top$ [GV13]. The library computes this with a one-sided Jacobi SVD (section 12) via `mv_nullvec`.

Example 4 (null vector by hand)**.** $\mathbf{A} = \left(\begin{smallmatrix} 1 & 2 \\ 2 & 4 \end{smallmatrix}\right)$ has rank 1 (row 2 = 2× row 1), so the smallest singular value is exactly 0 and the null vector solves $x_1 + 2x_2 = 0$: $\mathbf{x} = \frac{1}{\sqrt{5}}(2, -1)$. Check: $\mathbf{Ax} = \mathbf{0}$.

4 Camera model

A camera is a pinhole with intrinsic matrix

$$\mathbf{K} = \begin{pmatrix} f_x & s & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{pmatrix}, \tag{2}$$

rotation $\mathbf{R}$ and translation $\mathbf{t}$ mapping world points into the camera frame, $\mathbf{X}_c = \mathbf{RX} + \mathbf{t}$; the camera center is $\mathbf{C} = -\mathbf{R}^\top\mathbf{t}$. A world point projects to the homogeneous pixel

$$\mathbf{x} \;\simeq\; \mathbf{P}\begin{pmatrix} \mathbf{X} \\ 1 \end{pmatrix}, \qquad \mathbf{P} = \mathbf{K}\,[\,\mathbf{R} \mid \mathbf{t}\,] \tag{3}$$

[HZ04]. Real lenses deviate from (3); on normalized coordinates $(x, y) = (X_c/Z_c,\, Y_c/Z_c)$ with $r^2 = x^2 + y^2$ the Brown model gives the distorted

$$\begin{aligned} x_d &= x\,(1 + k_1 r^2 + k_2 r^4 + k_3 r^6) + 2p_1 xy + p_2(r^2 + 2x^2), \\ y_d &= y\,(1 + k_1 r^2 + k_2 r^4 + k_3 r^6) + p_1(r^2 + 2y^2) + 2p_2 xy, \end{aligned} \tag{4}$$

and the pixel is $\mathbf{K}(x_d, y_d, 1)^\top$. Back-projection inverts (4) by fixed-point iteration and returns the ray $\mathbf{C} + s\,\mathbf{d}$, $s > 0$.

Example 5 (projection)**.** Let $f_x{=}f_y{=}100$, $c_x{=}c_y{=}50$, $s{=}0$, $\mathbf{R} = \mathbf{I}$, $\mathbf{t} = \mathbf{0}$, no distortion, and $\mathbf{X} = (1, 2, 10)$. Normalized coordinates: $(x, y) = (1/10,\, 2/10) = (0.1, 0.2)$. Pixel: $u = 100 \cdot 0.1 + 50 = 60$, $v = 100 \cdot 0.2 + 50 = 70$ — the point lands at $(60, 70)$, cf. Example 1.

Example 6 (camera center)**.** Let $\mathbf{R} = \mathbf{R}_y(90°)$ from Example 2 and $\mathbf{t} = (3, 0, 0)$. Then $\mathbf{R}^\top\mathbf{t} = (0, 0, -3)$ and $\mathbf{C} = -\mathbf{R}^\top\mathbf{t} = (0, 0, 3)$: the camera sits three units down the world z axis. Check forward: $\mathbf{RC} + \mathbf{t} = (0{\cdot}0 + 0{\cdot}0 - 1{\cdot}3,\, 0,\, 0{\cdot}3) + (3, 0, 0) = (-3, 0, 0) + (3, 0, 0) = \mathbf{0}$ — the center maps to the camera origin, as it must.

4.1 The temporal dimension: a camera is also a clock

Equations (3)–(4) describe *where* a camera maps a world point; a complete model must also say *when*. Each camera runs on its own oscillator, so frame k (and, for a rolling-shutter sensor, row r within it) is captured at the world time

$$t_i(k, r) \;=\; \phi_i \;+\; k\,T_i \;+\; r\,\tau_i \;+\; \tfrac{1}{2}t_e, \tag{5}$$

where ϕ_i is the camera's clock offset, T_i its true frame period (consumer crystals deviate from nominal by 50–100 ppm, so T_i differs per camera and ϕ_i drifts), τ_i the row-readout time (zero for global shutter), and t_e the exposure whose midpoint is the effective sampling instant. The full camera state is therefore $(\mathbf{K}, \mathbf{R}, \mathbf{t},\ k_{1..5};\ \phi, T, \tau, t_e)$ — geometry *and* clock — and a measurement is not a pixel but a *timestamped* pixel: a ray with a time attached.

The multi-view machinery of section 6–section 8 silently assumes simultaneity: the epipolar constraint (10) and the triangulation stack (12) relate observations *of the same world state*. For static scenes the assumption is vacuous; for a point moving at velocity $\mathbf{v}$, a timestamp mismatch Δt between the paired observations displaces one of them by $\mathbf{v}\,\Delta t$ — producing epipolar residual and phantom depth in exactly the way quantified in Example 23. Simultaneity, like zero distortion, is thus an idealization to be *calibrated*: the clock parameters of (5) are estimable state, not assumptions — the display fixture measures them directly through a dedicated temporal calibration signal (subsection 5.8) and moving scene content re-estimates them at run time, with the measurement and correction practice developed in subsection A.3. When all cameras share a hardware trigger, ϕ, T collapse to common values and the temporal model disappears back into the geometric one — the trigger is, in this language, a way of buying (5) exactly rather than estimating it.

5 Calibration from planar targets

Calibration is part of the system, not an external input. Both supported targets are *planes with known metric structure*: a printed checkerboard page (subsection 5.5) and a controlled computer display (subsection 5.6). The mathematics is the same for both: several views of a known plane determine the intrinsics, the distortion, and each view's pose, by Zhang's method [Zha00, SM99] — the plane-based approach that displaced earlier calibration against machined 3D targets [Tsa87], paper and displays being the targets anyone can make.

5.1 The homography of a plane

Put the target plane at $Z = 0$ in its own frame. Then (3) collapses: writing $\mathbf{R} = [\mathbf{r}_1\,\mathbf{r}_2\,\mathbf{r}_3]$ (columns),

$$\mathbf{x} \simeq \mathbf{K}\,[\mathbf{r}_1\ \mathbf{r}_2\ \mathbf{t}]\,(X, Y, 1)^\top \ =\ \mathbf{H}\,(X, Y, 1)^\top, \tag{6}$$

a 3×3 *homography* with 8 degrees of freedom. Each point correspondence gives two linear equations in the entries of $\mathbf{H}$, so $n \geq 4$ points determine it as a homogeneous least-squares problem (Example 4 at scale $2n \times 9$), solved by normalized DLT (`mv_homography_dlt`; normalization as in section 12).

Example 7 (plane homography)**.** Frontal camera: $\mathbf{K}$ as in Example 5, $\mathbf{R} = \mathbf{I}$, $\mathbf{t} = (0, 0, 2)$. Then $[\mathbf{r}_1\,\mathbf{r}_2\,\mathbf{t}] = \left(\begin{smallmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 2 \end{smallmatrix}\right)$ and

$$\mathbf{H} = \mathbf{K}\begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 2 \end{pmatrix} = \begin{pmatrix} 100 & 0 & 100 \\ 0 & 100 & 100 \\ 0 & 0 & 2 \end{pmatrix} \simeq \begin{pmatrix} 50 & 0 & 50 \\ 0 & 50 & 50 \\ 0 & 0 & 1 \end{pmatrix}.$$

The plane point $(X, Y) = (0.1, 0.2)$ maps to $(50 \cdot 0.1 + 50,\ 50 \cdot 0.2 + 50) = (55, 60)$. Check via Example 5: the 3D point is $(0.1, 0.2, 2)$, normalized $(0.05, 0.1)$, pixel $(100 \cdot 0.05 + 50,\ 100 \cdot 0.1 + 50) = (55, 60)$.

5.2 Zhang's constraints and the closed-form intrinsics

Write $\mathbf{H} = [\mathbf{h}_1\,\mathbf{h}_2\,\mathbf{h}_3]$ (columns). From (6), $\mathbf{h}_1 \simeq \mathbf{K}\mathbf{r}_1$ and $\mathbf{h}_2 \simeq \mathbf{K}\mathbf{r}_2$ with the *same* scale. Since $\mathbf{r}_1, \mathbf{r}_2$ are orthonormal,

$$\mathbf{h}_1^\top \mathbf{B}\,\mathbf{h}_2 = 0, \qquad \mathbf{h}_1^\top \mathbf{B}\,\mathbf{h}_1 = \mathbf{h}_2^\top \mathbf{B}\,\mathbf{h}_2, \qquad \mathbf{B} \;\coloneqq\; \mathbf{K}^{-\top}\mathbf{K}^{-1}, \tag{7}$$

i.e. each view of the plane contributes two linear equations in the six independent entries of the symmetric matrix $\mathbf{B}$ (the *image of the absolute conic*). Three views generically determine $\mathbf{B}$ up to scale; two suffice if zero skew ($s{=}0 \Leftrightarrow B_{12}{=}0$) is imposed. The solution is again a null vector (Example 4); $\mathbf{K}$ then follows from $\mathbf{B}$ in closed form [Zha00], and the pose of each view is recovered as

$$\lambda = \frac{1}{\|\mathbf{K}^{-1}\mathbf{h}_1\|}, \quad \mathbf{r}_1 = \lambda\mathbf{K}^{-1}\mathbf{h}_1, \quad \mathbf{r}_2 = \lambda\mathbf{K}^{-1}\mathbf{h}_2, \quad \mathbf{r}_3 = \mathbf{r}_1 \times \mathbf{r}_2, \quad \mathbf{t} = \lambda\mathbf{K}^{-1}\mathbf{h}_3, \tag{8}$$

with $[\mathbf{r}_1\,\mathbf{r}_2\,\mathbf{r}_3]$ snapped to the nearest true rotation via SVD (noise makes it only approximately orthonormal).

Example 8 (pose from a homography, by hand)**.** Take the (unscaled) $\mathbf{H}$ of Example 7 and the known $\mathbf{K}$. Then

$$\mathbf{K}^{-1} = \begin{pmatrix} 0.01 & 0 & -0.5 \\ 0 & 0.01 & -0.5 \\ 0 & 0 & 1 \end{pmatrix}, \quad \mathbf{K}^{-1}\mathbf{h}_1 = \begin{pmatrix} 1 \\ 0 \\ 0 \end{pmatrix}, \quad \mathbf{K}^{-1}\mathbf{h}_2 = \begin{pmatrix} 0 \\ 1 \\ 0 \end{pmatrix}, \quad \mathbf{K}^{-1}\mathbf{h}_3 = \begin{pmatrix} 0 \\ 0 \\ 2 \end{pmatrix}.$$

Both constraint equations of (7) hold: $\mathbf{r}_1{\cdot}\mathbf{r}_2 = 0$ and $\|\mathbf{r}_1\| = \|\mathbf{r}_2\|$. The scale is $\lambda = 1/\|(1,0,0)\| = 1$, so (8) gives $\mathbf{r}_1 = (1,0,0)$, $\mathbf{r}_2 = (0,1,0)$, $\mathbf{r}_3 = \mathbf{r}_1 \times \mathbf{r}_2 = (0,0,1)$, $\mathbf{t} = (0,0,2)$ — exactly the frontal pose two metres from the target that we started from.

5.3 Radial distortion and alternation

With $\mathbf{K}$ and poses in hand, the ideal (undistorted) projection (u, v) of every target point is known, and (4) predicts the observed (u_d, v_d) linearly in (k_1, k_2):

$$u_d - u = (u - c_x)(k_1 r^2 + k_2 r^4), \qquad v_d - v = (v - c_y)(k_1 r^2 + k_2 r^4), \tag{9}$$

a least-squares problem with two unknowns. Since the homographies were fit to *distorted* points, the library alternates: fit geometry, estimate (k_1, k_2), undistort the observations, refit — a few rounds converge for moderate distortion [Zha00].

An identifiability caveat worth internalizing: over the small radius range a letter-page target covers, r^2 and r^4 are nearly proportional, so k_1 and k_2 are individually ill-determined even when their combined *curve* $k_1 r^2 + k_2 r^4$ — the thing that actually corrects pixels — is fit to well below noise (measured in section 13). Wide targets, or the full-field display target of subsection 5.6, are the cure when the individual coefficients matter.

5.4 Robust fitting from video: view selection

Filming the target instead of photographing it changes the statistics of the input in a way the estimator must respect. A one-minute video yields a hundred-plus decoded views, but they are not a hundred independent, well-chosen poses: most frames sit within a degree or two of

wherever the camera happened to dwell, and near-fronto-parallel views are exactly the singular configuration of plane-based calibration [SM99]. The linear system of subsection 5.2 weighs every view equally, so two hundred rows of noise-dominated near-degenerate constraints can outvote the handful of tilted frames that actually carry the focal length — and a single corrupted decode (a mislabeled corner lattice) poisons the fit outright. Both failure modes were first observed on real handheld videos (section 13), not in simulation: the full-video fit returned $f_x \approx 458$ at 35 px RMS while a hand-picked six-frame subset of the *same* video returned $f_x \approx 1599$ at 0.39 px.

The cure is classical: random sample consensus over *views* [FB81]. `mv_calib_planar_robust` (`mv/calib.h`) fits Zhang's closed form on 64 deterministic random six-view subsets; each candidate $\mathbf{K}$ is scored by the *median*, across all views, of the per-view reprojection RMS with that view's pose recovered from its own homography under the candidate $\mathbf{K}$. The median makes the score indifferent to a minority of corrupted views, while a degenerate subset produces a garbage $\mathbf{K}$ that fits almost *no* view and scores itself out. The winner defines inliers (per-view RMS within $3\times$ the best median); a final full fit with radial alternation runs on those inliers and is accepted only if it does not worsen the median score — so the answer is never worse than the best subset found. The seed is fixed: the same frames give the same calibration, byte for byte.

The unit test for this estimator (`test_calib_robust`, `tests/test_mv.c`) reproduces the field failure in simulation — 36 near-fronto-parallel views, four tilted ones, one corrupted, noise $\sigma = 0.3$ px — and demands the robust fit recover the focal length within 2% while plain Zhang, on the same data, fails outright. Building that test surfaced a trap worth recording: "corrupting" a view by *reversing* its point order corrupts nothing, because a checkerboard lattice is centro-symmetric and reversal is precisely a 180° rotation of the plane — still a perfectly valid homography. It is the same rotation ambiguity the pattern's M-array exists to resolve (subsection 5.9). A genuine corruption must be a non-projective scramble (the test swaps adjacent point pairs).

5.5 The printed target

The standard passive target is a checkerboard printed on a letter/A4 page: 8×10 squares of nominally 24 mm (192×240 mm), i.e. 7×9 inner corners (`mv/target.h`; `mv_target_render_pgm` emits the printable image — at 10 px/mm a 24 mm square is 240 px). Printers rescale, so the procedure is *print, then measure one square* and pass the measured size; only that measurement makes the calibration metric. Corner *positions* in the image are currently supplied by the caller (an external detector or manual marking); a saddle-point corner detector is on the roadmap (section 16). Ten to twenty views with strong tilt diversity (± 30–$40°$) condition the focal length well — near-frontal views leave it weakly observable, a property visible in the experiment of section 13.

5.6 The display as an active target

A computer display we control is a better target than paper wherever one is available: its pixel grid is a plane of exactly known geometry (pixel pitch from the panel datasheet, or measured once), and because we control what it shows *when*, correspondence needs no detector at all. The display plays, synchronized with capture, a sequence of binary stripe patterns that encode each display pixel's coordinate in *Gray code* [ISM84, Gen11]: bit b of the code is shown as stripes, followed by its inverse, for $b = 0..\lceil \log_2 W \rceil - 1$, then the same for the y axis. A camera pixel observing the display reads its bit as "brighter in the pattern or in the inverse?" — a per-pixel comparison immune to illumination, display brightness and gamma — and after all

frames has decoded *which display pixel it sees* (`mv/graycode.h`). This yields thousands of dense, uniformly spread correspondences per view; the same pipeline (subsection 5.1–subsection 5.3) consumes them, with plane coordinates = display pixel × pitch. Gray code rather than plain binary because adjacent stripes differ in exactly one bit: a threshold error at a stripe boundary displaces the decoded coordinate by at most one pixel instead of, e.g., 2^b.

This static sweep is operating mode M1 of the calibration subsystem (subsection 5.9).

Example 9 (Gray code by hand)**.** Encode $v = 6$: binary 110; Gray $= 110 \oplus 011 = 101$. Decode $g = 101$ by prefix-XOR: $b_2 = 1$, $b_1 = 1 \oplus 0 = 1$, $b_0 = 1 \oplus 0 \oplus 1 = 0$ — binary $110 = 6$. Now flip the last Gray bit (a boundary misread): $g' = 100 \to$ binary $111 = 7$, an error of exactly one position. In plain binary, flipping the top bit of 110 gives $010 = 2$, an error of four.

5.7 Design: the rotating display

For an N-camera rig (section 10) the planned calibration fixture is the display of subsection 5.6 mounted on a plate rotating at constant angular velocity ω, so that the screen sweeps past every camera each revolution. This subsection records the design; implementation is a roadmap item (section 16).

What the rotation provides. Three things, all substantial. (i) *Tilt diversity per camera*: as the screen sweeps past, every frame shows the known plane at a different orientation — exactly the varied-tilt view set Zhang's method needs, whose absence measurably degrades focal recovery (section 13). (ii) *One shared orbit*: every camera observes the same one-parameter family of poses — screen pose = mount offset composed with rotation about a fixed axis at constant ω. Fitting that orbit (axis, ω, phase, mount; ∼10 parameters) jointly across cameras links *all* of them into one world frame even if no two ever see the screen simultaneously: the orbit itself is the common reference object, replacing pairwise shared views. (iii) *The display is the clock*: the screen broadcasts its frame index, so every captured image self-reports display time and cameras synchronize to the rotation model without a hardware trigger.

What the rotation forbids. The static Gray-code stack of subsection 5.6 requires one camera pixel to observe one display point across ∼20 frames — impossible on a moving target. The pattern must therefore be *single-shot self-identifying*: every frame decodable alone. The design is layered:

- a checkerboard-type *saddle-point grid* as the metric layer — saddle locations are unbiased under perspective and under (moderate, symmetric) blur, unlike dot centroids, which both foreshortening and blur displace;
- *identity coding* embedded in the grid (marker bits in alternate squares, or an M-array whose every $k \times k$ window is unique), so partial and oblique views self-locate — most frames of a sweep are partial;
- a *frame-counter strip* along the border, Gray-coded *in time* (successive indices differ in one bit, cf. Example 9), providing the timestamp for the orbit model;
- a *temporal-calibration layer*: the counter timestamps are quantized to the display refresh, so a few dedicated *phase patches* toggle black/white every display frame — a camera whose exposure straddles a toggle records an intermediate gray level whose value is the

sub-refresh phase of its shutter (and, as a by-product, its true exposure duration). Counter + phase samples give every camera a stream of display-clock timestamps, from which each camera's clock *offset and rate* are fit — the display is not only the geometric reference but the rig's common clock (subsection A.3);

- *multi-scale structure* (a coarse grid nested with a fine one), since one display serves cameras at different distances and obliquities; strokes must span ≥3–4 pixels in the worst camera at the most oblique usable pose;
- *binary black/white only*: LCD gamma and contrast shift off-axis, so gray-level codes degrade with viewing angle while binary features with per-frame identity do not.

Across frames, only the identity layer changes (counter strip; optionally alternating coarse/fine emphasis) — pose variation is already supplied by the rotation. At low ω the screen moves <1 pixel between consecutive frames, so pattern-plus-inverse in consecutive frames remains usable and restores the per-pixel contrast robustness of subsection 5.6. The concrete arrangement of all layers on screen is specified once, in Figure 1 (subsection 5.9).

Example 10 (motion-blur budget)**.** Image-space blur during exposure t_e is $\omega\, r\, f\, t_e/Z$ pixels, with r the lever arm from axis to screen edge. For the experimental rig's numbers ($f = 800\,\text{px}$, $Z = 4.7\,\text{m}$) with $r = 0.25\,\text{m}$ and $\omega = 6\,\text{rpm} = 0.63\,\text{rad/s}$: the screen edge moves $0.63 \cdot 0.25 \cdot 800/4.7 \approx 27\,\text{px/s}$, so a 5 ms exposure blurs 0.13 px — comfortably sub-pixel, and between frames at 30 fps the screen shifts 0.9 px, validating the consecutive-inverse-frame option. The formula is the design knob linking ω, exposure, and screen brightness.

One mechanical trap: cant the screen. A screen standing parallel to the rotation axis presents orientations that all differ by rotation about a *single* axis — a weak/singular family for plane-based calibration [SM99], the same disease as the measured near-frontal weakness. Mounting the screen tilted back 20–30° makes its normal sweep a cone instead of a circle, exercising two rotation axes; this one bracket angle is the most important mechanical detail of the fixture.

Pipeline. Gate frames by obliquity (roughly 15–55° off-normal: frontal frames are uninformative, grazing frames unmatchable); per-frame homographies from the self-identifying grid; per-camera Zhang intrinsics from the gated set; joint orbit fit for all extrinsics + axis + ω + phase — the natural first customer for the planned Levenberg–Marquardt stage.

5.8 Temporal calibration: the display as a clock signal

Geometric calibration shows the cameras a known *spatial* structure; the temporal camera model (5) is calibrated the same way, by showing a known *temporal* structure — and the requirements mirror subsection 5.6 exactly: the signal must be *unambiguous* (coded, not periodic-only, or time aliases the way an uncoded stripe pattern aliases in space), *fine* (containing transitions faster than the precision sought), and *covering* (transitions must actually land inside camera exposures often enough to be sampled). Each clock parameter of (5) gets its own layer of the signal:

- *Offset ϕ, coarse:* the Gray-coded frame counter (subsection 5.7) — an absolute, self-delimiting time word, readable in any single frame, quantized to the display refresh T_d.

- *Offset ϕ, fine:* phase patches toggling every display frame. A camera exposure of length t_e that straddles a toggle integrates part old, part new value: the recorded gray level $g \in (0,1)$ *is* the toggle's position inside the exposure, refining the timestamp below T_d. A display cannot stagger toggles *within* a refresh — every change lands on the vsync — but the panel's own *scanout* staggers them for free: rows are updated top-to-bottom across the refresh, so a patch at row y toggles at vsync $+ (y/H)\,T_d$. Patches at several vertical positions therefore sample several sub-refresh phases (panel response time smears the edge; it is itself calibratable from the g statistics). Each patch straddles with probability t_e/T_d per frame; full coverage is not needed — the camera–display beat sweeps the phase, and a one-minute fit collects hundreds of straddle samples. One additional patch toggling at half rate disambiguates exposures longer than T_d.

- *Period T:* no extra signal needed — a linear fit of decoded display time against camera frame index over seconds (Example 24); the inevitable beat between camera and display rates is a *feature*, sweeping the sampling phase through the display cycle so all phases get observed without any genlock.

- *Row time τ (rolling shutter):* a full-screen toggle turns the rolling readout into its own measuring instrument — rows exposed before the toggle record one value, rows after it the other, so the captured frame shows horizontal *bands* whose pitch is the number of rows read per display period: $\tau = T_d/(\text{rows per band cycle})$, plus the readout direction for free.

- *Exposure t_e:* recovered jointly with the fine phase — the fraction of frames whose patches straddle, and the distribution of recorded gray levels, both depend on t_e alone once ϕ, T are known.

Example 11 (row time from banding)**.** A rolling-shutter camera photographs the display running a full-screen toggle at $T_d = 16.7\,\text{ms}$ (bright/dark half-periods of 8.33 ms). The captured frame shows bands 289 rows tall. Then $\tau = 8.33\,\text{ms}/289 \approx 28.8\,\mu\text{s}$ per row — and the full 1080-row readout takes $1080 \cdot 28.8\,\mu\text{s} \approx 31\,\text{ms}$, nearly the whole 33 ms frame period, which is typical: rolling sensors read continuously. One captured frame of a blinking screen measures the parameter that Example 22 showed corrupting the moving-target calibration — and once τ is known, every row's timestamp is known, turning the rolling-shutter *bias* into just more timestamped measurements per subsection 4.1.

What the display cannot fake: pose diversity. A tempting idea — render the pattern pre-warped, as if the plane were rotating, so nothing need physically move — fails for a fundamental reason worth recording. The camera observes the *physical screen plane*, whose pose is fixed: if the displayed pattern is warped by a chosen homography $\mathbf{W}_k$, the observed pattern-to-image map is $\mathbf{H}_k = \mathbf{H}_s\mathbf{W}_k$ with $\mathbf{H}_s$ the screen's one fixed physical homography. Since every $\mathbf{W}_k$ is known, the whole session carries exactly the information of the single view $\mathbf{H}_s$ — one homography, two Zhang constraints, $\mathbf{K}$ underdetermined (subsection 5.2); and feeding the $\mathbf{H}_k$ to the estimator as if they were real poses forces a solution unrelated to the camera's $\mathbf{K}$. Calibration information lives in the *physical* foreshortening of the light-emitting surface, and no amount of rendering synthesizes it. The practical consolations: (i) Zhang does not need *pure* rotation — any pose change serves, so inadvertent translation while repositioning is harmless; (ii) a laptop is its own rotation fixture: tilting the *lid* rotates the screen about the hinge axis, and swiveling the base on

the table provides the second axis — ten paused lid/swivel combinations are a complete Zhang set with no careful handling at all.

The same session that calibrates geometry therefore calibrates every clock parameter: one revolution of the fixture, read by all cameras, yields $(\mathbf{K}, \mathbf{R}, \mathbf{t}, k)$ from the corner grid and (ϕ, T, τ, t_e) from the temporal layers — the display is the rig's ruler and its metronome. Runtime maintenance of the clock model without the display (movers, LED beacon, golden pairs) is the practice material of subsection A.3; the concrete screen layout, operating modes, and data contract are concentrated in subsection 5.9.

5.9 The calibration subsystem: layout, modes, and data contract

This subsection is the *specification* that subsection 5.1–subsection 5.8 justify: the single contract against which the two calibration programs are written — the *display app* (renders the signal) and the *reader module* (consumes camera frames, emits records). Rationale lives in the sections cited; nothing here introduces a new idea. Implementation status: the pattern generator (`mv/pattern.h`), a synthetic renderer (`mv/render.h`), and the blind reader (`mv/reader.h`) are built and validated end-to-end in simulation (section 13); the display app exists as a self-contained web page (`tools/pattern.html`: modes M0/M2/M3, vsync-driven counter with missed-frame recovery, Retina-aware 1:1 device-pixel rendering, downloadable `DSP` log — any MacBook or iPhone becomes the fixture display; its embedded M-array is checked identical to the C constant), leaving the record-emitting wrapper, orbit fit, and clock fit.

Screen layout (Figure 1). One composite layout serves all single-shot work. Regions, sized by the rules already derived: white quiet border; a *version block* (static Gray-coded integer) naming the pattern-spec revision, so a reader never parses a layout it was not built for; the *frame-counter strip* (subsection 5.7), Gray-coded, changing every display frame; *phase patches* at several vertical positions (sub-refresh stagger via scanout, subsection 5.8); the central *saddle grid* with M-array identity marks in the squares (subsection 5.7), strokes $\geq$ 3–4 px in the worst camera at the worst usable obliquity (subsection 10.3); and one *nested fine grid* region for near/high-resolution cameras (subsection 5.7, multi-scale).

Pattern specification v1 (reference display 1920×1080). All coordinates in display pixels, origin top-left; the metric scale is the panel pixel pitch from the session manifest. Scaling to another resolution scales all entries proportionally, and bumps the version.

Region	Geometry (px)	Content
quiet border	32 all around	white
version block	origin (32,32), 10 cells of 48×48	marker $[1,0]$ + 8 Gray bits, static
nested fine grid	origin (560,32), 3×3 cells of 40	plain checkerboard
phase patches P1–P4	96×96 at (912,32), (1792,32), (32,472), (1792,472)	toggle every frame; scanout phases
phase patch P5	96×96 at (32,860)	toggles every 2nd frame
saddle grid	origin (192,160), 19×10 cells of 80	checkerboard + mark dots
M-array marks	24×24 dot at each cell center	bit = dot present (contrast-inverted)
corner lattice	$18 \times 9 = 162$ corners	id $= 18j + i$, origin nearest version block
counter strip	origin (192,984), 24 cells of 64×64	$[1,0]$ + 20 Gray bits + parity, $\overline{\text{parity}}$

Table 2: Pattern spec v1. A 20-bit counter at 60 Hz wraps in ≈ 4.9 h.

M-array construction. One bit per grid square ($19 \times 10 = 190$ bits): a black center dot on white squares, a white dot on black squares, so every square carries a bit and no dot approaches a saddle corner (≥ 28 px clearance). The window size must be 4×4: with 3×3 windows the 136 windows $\times 4$ rotations demand 544 distinct 9-bit codes and only 512 exist — pigeonhole forbids it. At 4×4, the 112 windows $\times 4$ rotations need 448 of 2^{16} codes, ample slack. The array is found once by seeded search with greedy repair (seed 20260803, solved in 30 steps) and shipped as a constant table in `libmv` (`src/pattern.c`), with the exhaustively verified properties: all 448 rotated window codes are pairwise distinct with Hamming distance ≥ 2. The rotation clause is what resolves the checkerboard's 4-fold orientation ambiguity: any single decoded window fixes the global orientation, after which corner ids are assigned in the pattern frame.

Reader algorithms. Fixed choices, so two implementations decode identically: *binarization* — local mean over a 31×31 box (integral image) with ±hysteresis; black/white radiometric references taken per region from the quiet border and the nearest grid cells. *Saddle detection* — correlation with a 4-phase saddle template bank ($r = 5$ px), candidates by non-maximum suppression, sub-pixel by 2D quadratic fit on the 3×3 response neighborhood (the same parabola rule as the stereo matcher). *Grid assembly* — seed at the strongest saddle, estimate a local homography from its four lattice neighbors, grow breadth-first, predicting each next corner through the local homography and accepting within 0.3 cell. *Decoding* — sample mark presence at cell centers through the local homography against the local black/white references; look up each 4×4 window (and its rotations) in the shipped table; majority vote fixes position and orientation. *Counter/patches* — sample cell and patch centers the same way; counter valid iff markers and both parity cells check; its `conf` field is the minimum sampling margin $\min_i |g_i - \frac{1}{2}| \cdot 2 \in [0, 1]$. `PHS` records are emitted only for $0.15 < g < 0.85$ (certain straddle). *Banding* (mode M3) — column-median intensity profile per frame; transition rows at gradient maxima; `BND` pitch = median inter-transition spacing.

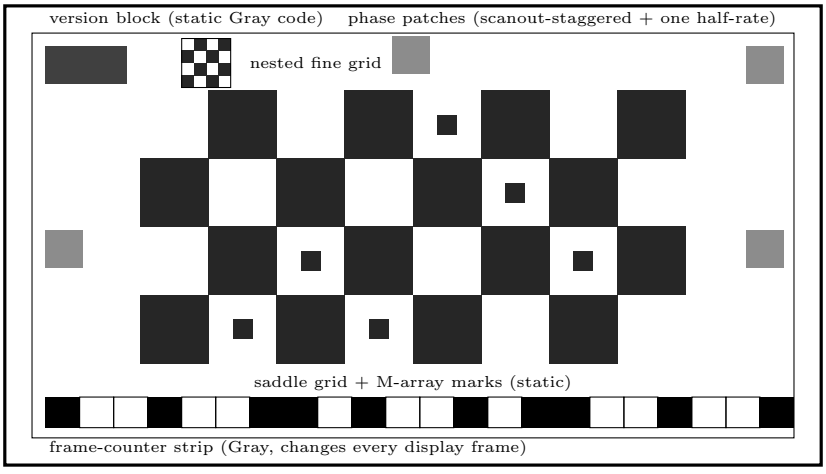


Figure 1: Composite screen layout (mode M2; schematic, not to scale). Geometric layers are static; only the counter strip and phase patches change per display frame.

Display-app modes. The app is a mode machine; a calibration session is a scripted sequence of modes.

Mode	Screen shows	Plate	Calibrates	Rationale
M0	flat gray ladder (0–100%)	any	radiometric transfer, flat field	§A.4
M1	Gray-code stripe sweep + inverses	parked	dense correspondences → $\mathbf{K}$, pose	§5.6
M2	composite layout (Figure 1)	ω const.	$\mathbf{K}, \mathbf{R}, \mathbf{t}, k$; orbit; ϕ, T	§5.7, §5.8
M3	full-screen toggle at T_d	parked	τ (banding), t_e	Ex. 11

Table 3: Display-app operating modes.

Reader output: the record schema. The reader consumes raw frames per camera and emits ASCII records, one per line — the same recordable/replayable oracle pattern as the inference sidecar (subsection 15.2); recorded sessions are the test fixtures.

Record	Fields	Emitted	Consumed by
`VER`	spec version, display px, pitch mm, T_d	session start	reader/estimator config check
`FRM`	cam, frame k, arrival time	every frame	bookkeeping, drift diagnostics
`CTR`	cam, k, counter value, confidence	when strip decodes	clock fit (ϕ, T); orbit phase
`PHS`	cam, k, patch id, gray level	straddled patches	clock fit (fine ϕ); t_e
`CRN`	cam, k, corner id, u, v	per detected corner	homographies → Zhang, orbit fit
`BND`	cam, k, band pitch, direction	mode M3	row time τ
`DSP`	display frame k, host time	display app, per frame	clock cross-check

Table 4: Calibration record schema (all ASCII, one record per line).

Field-level grammar: whitespace-separated `key=value` after the record tag; times are seconds as decimal doubles on the host `CLOCK_MONOTONIC`; image coordinates are *raw distorted* pixels (undistortion is the estimators' job); corner `id` is the pattern-frame lattice index of Table 2. Example lines:

```
VER spec=1 w=1920 h=1080 pitch_mm=0.2745 Td_ms=16.667
FRM cam=0 k=1234 t=5182.30412
CTR cam=0 k=1234 count=48213 conf=0.94
PHS cam=0 k=1234 patch=2 g=0.371
CRN cam=0 k=1234 id=57 u=812.431 v=402.117
BND cam=0 k=87 pitch=289.3 dir=down
DSP k=309882 t=5182.29561
```

Session manifest and capture interface. Each session directory holds a manifest (`key=value` text): rig id; per camera — id, serial, driver, nominal fps, lens focal; display — resolution, pixel pitch, T_d; plate — nominal ω, cant angle; all RNG seeds. The capture layer, shared verbatim with the inference sidecar (subsection 15.2), is a three-call C contract: `open(device)`, blocking `get(frame)`, `close()`, where a frame is 8-bit mono pixels + width/height/stride + camera id + driver sequence number + arrival time (`CLOCK_MONOTONIC` at driver dequeue — the timestamp whose limitations subsection A.3 exists to overcome). Recorded streams are PGM sequences plus an index of `FRM` lines; a recorded session replays bit-identically through reader and estimators.

Estimator definitions. *Orbit fit*: unknowns are the plate axis (unit vector, 2 dof), a point on the axis (2 dof), ω, phase ϕ_0, the fixed display-to-plate mount (6 dof), and each camera's pose (6 dof each); the display pose at time t is the mount composed with rotation about the axis by $\omega t + \phi_0$, and the cost is total corner reprojection error over all cameras and gated frames, Huber-robustified at 1 px. Initialization: per-frame Zhang extrinsics (subsection 5.2)

give a display-center trajectory per camera; its plane normal initializes the axis, its circle fit (subsection 14.3's known machinery) the center, and the `CTR` timeline ω, ϕ_0. *Clock fit*: per camera, Theil–Sen (deterministic median-of-slopes) on ($\texttt{CTR.count} \cdot T_d$) vs frame index rejects counter misdecodes, a 3-MAD gate then admits samples to least squares for (ϕ_i, T_i), and `PHS` straddle samples jointly refine ϕ_i and t_e against the scanout-phase model of subsection 5.8. Acceptance bands for both are the subsection 13.2 machinery evaluated at the rig's *measured* noise (subsection A.4), not at the synthetic $\sigma = 0.3$ px.

Display-app guarantees. Fullscreen exclusive; compositor scaling disabled and verified optically (a single-pixel checker in the quiet border must alias to gray in any camera — if it resolves, the path is not 1:1); all emitted values pure 0 or 255 (gamma-immune); counter incremented in the vsync callback, and after a missed vsync advanced by the number of elapsed refreshes; the app logs its own `DSP` record per displayed frame as the display-side ground truth. The pattern generator itself lives in `libmv` (emitting frames the app only blits), so every layer of Table 2 is unit-testable against the reader without a display.

Session choreography and acceptance. A session is: M0 ladder → M3 (∼30 frames) → M2 for N revolutions (→ M1 per camera only if the plate can park facing it); then the estimator chain runs: per-camera Zhang on obliquity-gated M2 frames → joint orbit fit → clock fit → radial alternation. The session is *accepted* only if the standing statistics land in their bands (subsection 13.2): reprojection T5 ratio within its documented band; clock-fit residuals within the Example 24 budget; banding-derived τ consistent across repeats; and the resulting analytic $\mathbf{F}$ stored as the baseline for the run-time drift monitor (section 6). Roadmap items 1–2 (section 16) are precisely the implementation of this subsection.

5.10 The coarse tier (pattern spec v2)

The fine layout above assumes the camera can resolve a 24 px identity dot; the round-four field experiment (section 13) measured where that assumption dies — about 0.6 m for a 720p webcam — and deployment cameras watch rooms, not desktops. The coarse tier is the same display speaking more slowly: a 6×3 checkerboard of 270 px cells (10 inner corners), an asymmetric L of three 90 px orientation marks whose cell set maps to a disjoint set under every non-identity rotation, and a 12-cell counter strip ([1,0] sync, 8 Gray bits, parity, inverted parity) rendered as *isolated* 90 px squares. Every feature is ∼3.4× the fine tier's, so (19) stretches by the same factor: ∼2 m for the webcam, ∼3.7 m for a phone. The price is density (10 correspondences per frame instead of 162) and counter width (8 bits wraps every 256 refreshes ≈ 4.3 s; consumers disambiguate by capture-order proximity, as `tools/rigcalib.c` does, or by rig-rigidity consistency). Generator: `mv_pattern2_render`; reader: `mv_read_coarse` (escalates over 2×/4× downsampling for close views); display app: mode M4.

Two implementation lessons, both found by the sim harness before any field time was spent. First, *gutter saddles*: two dark squares separated by a thin light gap form a genuine saddle point between them — black left and right, white above and below — indistinguishable locally from a checkerboard corner; a counter strip of adjacent solid cells decorates the image with a full row of them. Isolating the squares only moves the saddles into the gutters; the working cure is geometric placement (the strip sits 1.5 lattice steps below the bottom corner row, so integer-step lattice growth cannot reach it) *plus* tolerance in the reader. Second, *keystone defeats layout*

spacing: under a 20° tilt the projective foreshortening moves the strip from 1.5 to ∼1.35 image-steps — back inside the capture gate — so no static layout margin survives all poses. The reader therefore does not demand a clean 5×2 bounding box: it grows generously, enumerates candidate 5×2 windows inside the grown lattice, and lets the exact validator (all 18 cell colors, all 18 mark states, sync and double parity) decide. The validation round-trip test (`test_pattern_coarse`) runs at exactly the geometry that defeated the fine tier in the field — a 720p camera with the pattern 160 px wide — plus a 180°-rolled, 20°-tilted, noisy view, and demands all ten corners, correct ids, sub-pixel RMS, and an exact counter.

6 Two-view epipolar geometry

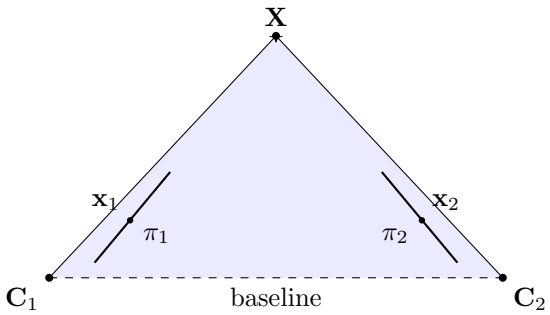


Figure 2: Epipolar geometry: the camera centers $\mathbf{C}_1, \mathbf{C}_2$ and the scene point $\mathbf{X}$ span the epipolar plane (shaded); its intersections with the image planes π_1, π_2 are the corresponding epipolar lines on which $\mathbf{x}_1$ and $\mathbf{x}_2$ must lie.

Corresponding pixels of the two views are linked by the fundamental matrix $\mathbf{F}$ through the epipolar constraint

$$\mathbf{x}_2^\top \mathbf{F}\, \mathbf{x}_1 = 0, \tag{10}$$

with $\mathbf{F}$ of rank 2; the line $\boldsymbol{\ell}_2 = \mathbf{F}\mathbf{x}_1$ is the locus in image 2 of possible matches of $\mathbf{x}_1$ (Figure 2). For *calibrated* views the essential matrix $\mathbf{E} = \mathbf{K}_2^\top \mathbf{F}\, \mathbf{K}_1 = [\mathbf{t}]_\times \mathbf{R}$ carries the same constraint in normalized coordinates [LH81]; note the reappearance of (1).

Because our rig is calibrated, $\mathbf{F}$ is available in closed form,

$$\mathbf{F} = [\, e_2 \,]_\times \, \mathbf{P}_2 \, \mathbf{P}_1^{+}, \qquad e_2 = \mathbf{P}_2 \begin{pmatrix} \mathbf{C}_1 \\ 1 \end{pmatrix}, \tag{11}$$

where $\mathbf{P}_1^{+}$ is the pseudo-inverse of $\mathbf{P}_1$ [HZ04]. The system computes (11) once at start-up and uses it for *correspondence gating*: any candidate match whose symmetric epipolar distance exceeds a threshold is rejected before triangulation.

Independently, $\mathbf{F}$ can be *estimated* from $n \geq 8$ correspondences by the normalized 8-point algorithm [Har97]: each pair turns (10) into one linear equation in the nine entries of $\mathbf{F}$ — again the homogeneous least-squares pattern of Example 4 — followed by rank-2 enforcement via SVD. In this system the estimator serves two purposes: (i) *health monitoring* — if the estimate from live correspondences drifts from the calibrated (11), the rig has been bumped or the calibration is stale; (ii) bootstrapping geometry for *uncalibrated* auxiliary photos (section 11). On contaminated correspondence sets the estimator is wrapped in a RANSAC loop [FB81] (planned with the feature layer).

Example 12 (essential matrix of a rectified rig)**.** Two identical cameras, the second displaced one unit along x: $\mathbf{R} = \mathbf{I}$, $\mathbf{C}_2 = (1, 0, 0)$, so $\mathbf{t} = -\mathbf{R}\mathbf{C}_2 = (-1, 0, 0)$. By Example 24,

$$\mathbf{E} = [\mathbf{t}]_\times \mathbf{R} = \begin{pmatrix} 0 & 0 & 0 \\ 0 & 0 & 1 \\ 0 & -1 & 0 \end{pmatrix}, \qquad \mathbf{x}_2^\top \mathbf{E}\, \mathbf{x}_1 = (x_2, y_2, 1) \begin{pmatrix} 0 \\ 1 \\ -y_1 \end{pmatrix} = y_2 - y_1.$$

The epipolar constraint degenerates to $y_2 = y_1$: corresponding points lie on the *same image row*. This is precisely the situation rectification (section 8) manufactures for arbitrary rigs.

7 Triangulation

Given observations $\mathbf{x}^{(v)} = (u_v, w_v)$ of the same point in $N \geq 2$ views with projection matrices $\mathbf{P}^{(v)}$, each view contributes two linear equations obtained by eliminating the projective scale from (3):

$$\begin{pmatrix} u_v\, \mathbf{P}^{(v)}_{3,:} - \mathbf{P}^{(v)}_{1,:} \\ w_v\, \mathbf{P}^{(v)}_{3,:} - \mathbf{P}^{(v)}_{2,:} \end{pmatrix} \begin{pmatrix} \mathbf{X} \\ 1 \end{pmatrix} = \mathbf{0}, \tag{12}$$

and stacking all views gives a $2N \times 4$ homogeneous system solved, once more, as in Example 4 — the DLT triangulation [HZ04]. The implementation accepts any $N \geq 2$, so still photographs registered into the rig frame (section 11) simply extend the same call; what additional views and additional pixels buy, quantitatively, is analyzed in section 10. DLT minimizes an algebraic residual; the reprojection-optimal two-view method of Hartley–Sturm [HS97] is a planned refinement, worthwhile mainly for wide-baseline auxiliary views. The library reports the RMS reprojection error of every triangulated point — the quantity an eventual bundle adjustment would minimize [TMHF00].

Example 13 (triangulation by similar triangles)**.** Rig of Example 12 with $\mathbf{K}$ of Example 5 ($f = 100$, $c = 50$) and baseline $B = 0.2$: camera 2 at $(0.2, 0, 0)$. Scene point $\mathbf{X} = (0.1, 0, 2)$. View 1: $(x, y) = (0.05, 0)$, pixel $u_1 = 100 \cdot 0.05 + 50 = 55$. View 2: camera coordinates $(0.1 - 0.2,\ 0,\ 2) = (-0.1, 0, 2)$, so $u_2 = 100 \cdot (-0.05) + 50 = 45$. Disparity $d = u_1 - u_2 = 10$, and (13) gives $Z = fB/d = 100 \cdot 0.2/10 = 2$ ✓. The lateral coordinate follows from view 1: $X = Z\,(u_1 - c_x)/f = 2 \cdot 5/100 = 0.1$ ✓— the full point recovered by hand.

8 Rectification and dense stereo

For dense per-pixel depth the pair is first *rectified*: both cameras are rotated (synthetically, about their centers) onto a common orientation whose x axis is the baseline, making the geometry of Example 12 true by construction — epipolar lines become identical image rows and correspondence search collapses to 1D. We use the compact algorithm of Fusiello, Trucco and Verri [FTV00]: the new rotation has rows $(\mathbf{x}, \mathbf{y}, \mathbf{z})$ with $\mathbf{x}$ the unit baseline, $\mathbf{y} = \mathbf{k} \times \mathbf{x}$ ($\mathbf{k}$ the old optical axis of camera 1) and $\mathbf{z} = \mathbf{x} \times \mathbf{y}$; both views share the averaged intrinsics with zero skew, and the pixel maps are the homographies $\mathbf{H}_i = \mathbf{K}_{\text{new}} \mathbf{R}_{\text{new}} \mathbf{R}_i^\top \mathbf{K}_i^{-1}$ (homographies again — cf. (6), here induced by pure rotation instead of a scene plane). Since the rig is stationary, $\mathbf{H}_1, \mathbf{H}_2$ are computed once and each video frame is warped by table lookup.

On the rectified pair, dense correspondence is found by block matching: for each left pixel the sum of absolute differences (SAD) over a $(2w{+}1)^2$ window is minimized along the corresponding

right row over a disparity range, followed by sub-pixel refinement by parabolic interpolation of the cost around the minimum. This is the classic local baseline method in the taxonomy of Scharstein and Szeliski [SS02] — chosen deliberately: it is simple, portable, trivially parallel, and its failure modes (textureless regions, occlusions, repetitive patterns) are well understood, giving a controlled baseline against which future matchers (rank/census transforms, semi-global aggregation) can be measured within the same harness.

Matched disparity d converts to metric depth by

$$Z = \frac{f\,B}{d}, \tag{13}$$

with f the shared rectified focal length (pixels) and B the baseline. Differentiating (13) gives the expected depth uncertainty

$$\sigma_Z \;=\; \frac{Z^2}{f\,B}\,\sigma_d \tag{14}$$

for disparity noise σ_d — the quadratic growth with Z is the fundamental accuracy law of the rig.

Example 14 (depth error budget)**.** The experimental rig of section 13: $f = 800\,\text{px}$, $B = 0.5\,\text{m}$, mean depth $Z = 4.7\,\text{m}$, per-coordinate noise $\sigma = 0.3\,\text{px}$ so $\sigma_d = \sqrt{2}\,\sigma \approx 0.42\,\text{px}$. Then $\sigma_Z = 4.7^2 \cdot 0.42/(800 \cdot 0.5) \approx 0.023\,\text{m}$: about 2.3 cm of depth uncertainty — compare the measured 2.65 cm RMS 3D error in Table 5.

9 From depth to models

Every depth estimate back-projects to a 3D point in the fixed world frame; points are accumulated in a growable cloud with optional per-point color and exported as ASCII PLY — the text variant of the polygon file format: a self-describing header followed by one line per vertex, directly loadable in MeshLab, CloudCompare or Blender. A complete two-point colored cloud, exactly as `mv_cloud_write_ply` emits it:

```
ply
format ascii 1.0
element vertex 2
property float x
property float y
property float z
property uchar red
property uchar green
property uchar blue
end_header
0.1 0.25 4.7 40 200 40
0.12 0.25 4.71 220 50 50
```

The header names each element type and its properties in order; every vertex line then lists those property values — trivially writable from C99, diffable, and universally read. This section maps out what can be built *on top* of that cloud: the representations worth targeting, how robust each is in the literature, and the background/foreground architecture the stationary rig makes natural.

9.1 The representation ladder

Model types form a ladder of increasing geometric commitment; each rung is derived from the one below, and robustness generally *decreases* upward:

1. *Point clouds* (implemented): no topology, no commitments; ideal for measurement and inspection; never wrong, never finished.
2. *Occupancy / signed-distance volumes*: the truncated signed distance function (TSDF) fuses many noisy depth maps by running per-voxel averages [CL96]; incremental, noise-averaging, topology-agnostic — the workhorse of the KinectFusion lineage [NIH+11] and the natural accumulation structure for our per-frame depth (elaborated in subsection 9.2).
3. *Triangulated meshes*: extracted from a TSDF by marching cubes [LC87], or from oriented points by Poisson reconstruction [KBH06]. Both are mature and reliable *given* rung 2; meshing raw noisy stereo clouds directly (Delaunay-based, ball-pivoting) is fragile and best avoided.
4. *Geometric primitives* ("something more geometric"): RANSAC plane extraction [FB81] is the single most robust operation in the entire reconstruction literature — floors, walls, table tops fall out almost for free; cylinders, spheres and cones follow with efficient RANSAC variants [SWK07] at the cost of tuning; box/cuboid fits give furniture-level abstraction. Beyond quadrics, parametric fitting becomes brittle.
5. *Object/semantic models* (chair, table, person): robust today only via learned detectors — outside the dependency-free core, but the geometry below is exactly what such an external module would consume.

Modern learned implicit representations (neural radiance fields, Gaussian splatting) excel at appearance and view synthesis but are not metrically superior for measurement, and sit outside this system's portable-C99, controlled-environment charter.

9.2 The TSDF, in detail

Since rung 2 is where this system's model extraction will live, it deserves more than a name — starting with its plumbing.

Inputs, state, outputs. The TSDF's *input* is a stream of calibrated depth maps: per camera and frame, a depth value per pixel (from the dense stereo of section 8, disparity → depth via (13)) together with that camera's fixed calibration. Its *state* — the TSDF itself — is a voxel grid over the working volume holding two numbers per voxel. Its *outputs* are three: a triangulated mesh (the "proper model," extracted on demand, exported as PLY); a queryable distance field (how far is any point from the nearest surface — subsection 9.3's foreground test is exactly "does this new depth sample disagree with D?"); and, by raycasting the field, synthetic depth/normal maps from any viewpoint [NIH+11]. Build-side and use-side data flow:

$$\underbrace{\text{depth maps} + \text{calibration}}_{\text{per frame}} \rightarrow \underbrace{\text{voxel sweep (15)}}_{\text{fusion}} \rightarrow \underbrace{(D, W)\ \text{grid}}_{\text{state}} \rightarrow \begin{cases} \text{march. cubes} \rightarrow \text{mesh} \\ \text{queries} \rightarrow \text{foreground} \\ \text{raycast} \rightarrow \text{depth} \end{cases}$$

Fusion is *incremental and order-insensitive* — the update (15) below is a running average, so frames can arrive in any order, streams from the two cameras interleave freely, and there is never a batch step; the mesh is extracted whenever wanted without interrupting accumulation.

The fusion interface: enriched samples. A design decision worth recording: the TSDF is *not* a function of a bare point cloud. Two of its essential ingredients are properties of the observing *ray*, not of the point — the *sign* of distance (which side of the surface a voxel is on) and *carving* (the measured emptiness of the space between camera and surface). A cloud that has discarded viewpoints forces the fragile normals-estimation path (estimate normals by local fits, then propagate a consistent orientation globally [HDD+92] — machinery that exists precisely because provenance was thrown away). The fix is one field, not a parallel pipeline: the fusion interchange type is the *enriched sample*

$$(\text{point},\ \text{origin},\ \text{weight},\ \text{time}),$$

and the TSDF is a pure, commutative fold over a stream of such samples — origin reconstructs the ray, sign, carving segment, and the inverse-variance weight; *time* is the same per-measurement timestamp the temporal camera model (subsection 4.1) already demands. On a stationary rig the origin compresses to a camera id. This is standard practice, in three forms across the literature: per-scan viewpoints attached to clouds (the PCD `VIEWPOINT` field, E57 scan poses, and OctoMap's `insertPointCloud(cloud, origin)` API, whose free-space carving is exactly ours [HWB+13]); per-point attributes in scanning formats; and, at the fully-enriched limit, *surfel maps* [KLL+13], where the enriched point is itself the fused model. The canonical TSDF papers [CL96, NIH+11] are instead depth-map-native — equivalent content, dense packaging — and this system takes both: the sample fold defines the semantics (so sparse feature points, dense depth pixels, and future laser returns, subsection 11.3, all fuse through one interface), while the precomputed voxel-major sweep below is the fast path when the producer happens to be a dense static-camera depth map.

The state and its update. A *truncated signed distance function* discretizes the working volume into voxels; each voxel v stores two numbers, a distance estimate $D(v)$ and a weight $W(v)$. When a depth map arrives, every voxel looks up the depth Z_{obs} at the pixel it projects to, compares it with its own depth Z_v along that camera ray, and forms the signed offset $\eta = Z_{\text{obs}} - Z_v$ — positive if the voxel lies in front of the observed surface, negative behind it. The observation is *truncated*, $d = \max(-\tau, \min(\tau, \eta))$, and fused by a weighted running average [CL96]:

$$D \leftarrow \frac{W\,D + w\,d}{W + w}, \qquad W \leftarrow W + w, \tag{15}$$

with voxels far *behind* the surface ($\eta < -\tau$) left untouched — a voxel is only carved by rays that pass near or through it. The reconstructed surface is the zero level set of D, extracted as a mesh by marching cubes [LC87] with linear interpolation along voxel edges for sub-voxel placement.

Three design choices connect (15) to the rest of this document. *Truncation* τ should cover the depth noise: $\tau \approx 3\sigma_Z$ at the working depth ($\approx 7\,\text{cm}$ for our rig at $4.7\,\text{m}$, Example 14) — larger wastes the band on uninformative updates, smaller throws away genuine measurements. *Weights* implement the fusion discipline of subsection 10.4: $w = 1/\sigma_Z^2(Z_{\text{obs}})$, so near observations (and, later, laser samples, subsection 11.3) dominate exactly in proportion to their information. And averaging *in distance space* is what makes the TSDF better than averaging points: each

voxel's D is a $\sqrt{n}$-averaged estimate of the same physical quantity, so surface precision improves with observation count — per-frame $\sigma_Z = 23\,\text{mm}$ becomes a sub-millimetre statistical floor within a minute of video ($23/\sqrt{900} \approx 0.8\,\text{mm}$), long before which systematic terms take over (subsection 13.1).

The stationary rig adds one more gift: because the cameras never move, each voxel's projection into each camera — the pixel it reads and its depth along the ray — is *constant*, computable once at start-up and stored. The per-frame update collapses to a table-driven sweep: one depth-map lookup, one compare, one multiply-add per voxel per camera.

Example 15 (TSDF budget)**.** The working volume ($2.0 \times 1.3 \times 2.1\,\text{m}$) at 1 cm voxels: $200 \times 130 \times 210 \approx 5.5$M voxels; at 6 bytes each (float D + 16-bit W) $\approx 33\,\text{MB}$ — trivial. At 5 mm: 44M voxels, $\approx 260\,\text{MB}$ — still a desktop, not a cluster. The per-frame sweep at 1 cm and 30 fps costs 5.5M × 2 cameras $\times 30 \approx 330$M table-driven updates per second — feasible in plain C, and reducible by an order of magnitude by updating only the truncation band around the current surface. Sub-voxel zero-crossing interpolation means the 1 cm grid does not limit surface accuracy to 1 cm; the averaged noise does, and it is far smaller.

9.3 Background extraction: the easiest product

For a stationary rig, the *static scene* — walls, floor, furniture — is the most robustly extractable model, because time works entirely in its favor: the per-pixel *temporal median* of depth over minutes or hours is immune to anything that moves through the scene (the same robust-statistics trick as classical 2D background subtraction, lifted to depth), and every additional frame averages noise down. The pipeline is: temporal-median depth map → TSDF fusion [CL96] → marching-cubes mesh [LC87] of the static scene → RANSAC planes for floor/walls/table-tops → primitive fits [SWK07] on the remaining clusters for furniture-level geometry. Every stage is rung-1–4 robust. The background model is also *self-maintaining*: a piece of furniture moved to a new position is a persistent change — detected exactly like a lamp event in section 14, but in geometry — triggering a local update and, as a by-product, an event log of scene rearrangements.

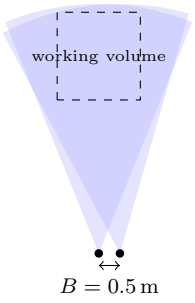
9.4 Separating the movers

With a background model in hand, foreground extraction is subtraction: any depth estimate (or triangulated point) that disagrees with the background beyond its noise band belongs to a *mover*. Foreground points are clustered by spatial proximity per frame, clusters are tracked over time (section 14 implements exactly this machinery), and classification into insect / robot / human / other falls out of crude but effective geometric statistics — extent, height above floor, speed, and persistence: a single isolated point moving at 0.1–1 m/s is an insect; a ∼0.3 m-wide cluster at constant ∼0.1 m height gliding on the floor plane is the robot; a 0.4–0.6 m cluster extending 1.5–2 m above the floor is a person. The scenario studies of section 14 quantify each of these tracks separately; the unified pipeline — background subtraction feeding one multi-target tracker with per-class models — is the integration goal they build toward.

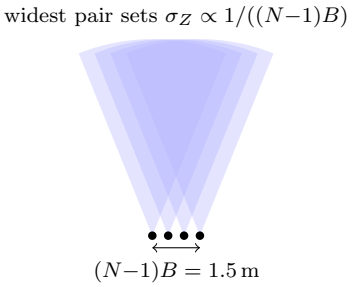
10 Scaling the rig: more cameras and more pixels

The two-camera rig is the minimum that triangulates. This section works out what changes when the rig grows — N cameras around the same volume, or higher (possibly unequal) resolutions

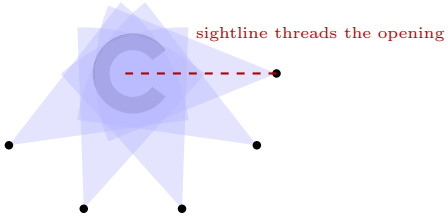
— so that hardware decisions can be made from the error laws instead of intuition. The library is already shaped for this: `mv_triangulate` accepts any $N \geq 2$, and every camera carries its own $\mathbf{K}$, so heterogeneous rigs need no new code paths in the sparse pipeline. Figure 3 shows the four camera-positioning scenarios this section analyzes, with each camera's viewing frustum drawn to the rig's actual field of view ($2\arctan(320/800) \approx 44°$ horizontally for the reference cameras).



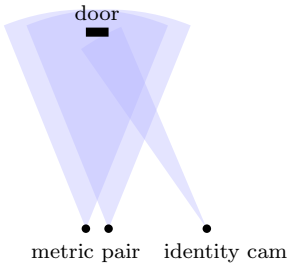
(a) reference pair (section 1): 6° convergence, overlap covers the volume



(b) collinear multi-baseline array (subsection 10.1, Example 16): short pairs match, wide pair measures



(c) coverage ring for occlusion and topology (subsection 10.2): spread beats baseline; one camera must see through the handle



(d) heterogeneous rig (subsection 10.4, subsection 15.3): wide pair for geometry, narrow frustum for face pixels at the chokepoint

Figure 3: Camera-positioning scenarios, top-down, frustums drawn as viewing wedges (range truncated for the page; the real frusta extend with distance). Overlap density shows where triangulation is possible.

10.1 N cameras: accuracy

Adding cameras improves triangulation through two distinct mechanisms.

Geometry (the dominant effect). The depth law (14) is governed by the baseline: rays from nearby centers intersect at grazing angles, and the intersection point slides far along the rays for small image perturbations. With cameras spread out, the *widest* usable pair sets the leading term: for N collinear cameras spaced B apart, the widest baseline is $(N-1)B$ and

$$\sigma_Z^{\text{widest}} = \frac{Z^2}{f\,(N-1)B}\,\sigma_d, \tag{16}$$

an $(N-1)$-fold improvement before any averaging. The catch is correspondence: wide pairs see the surface from very different angles, so matching is harder — which is exactly why multi-baseline stereo [OK93] matches *short* pairs (easy, unambiguous) and accumulates their evidence in a common inverse-depth parameterization, getting wide-pair accuracy at short-pair matching difficulty. Repetitive texture that fools one pair (a periodic fence aliases at one baseline) is disambiguated by any other baseline whose period does not divide it — the match must be consistent across *all* pairs simultaneously.

Redundancy. With visibility and geometry held fixed, N views of the same point give the usual statistical $\sqrt{N}$: the DLT stack (12) grows to $2N \times 4$ rows and the estimate averages the per-view noise. This is the smaller effect — doubling N buys $\sqrt{2} \approx 1.4\times$; doubling the *spread* buys $2\times$.

Example 16 (four collinear cameras)**.** Extend the experimental rig of Example 14 ($f = 800\,\text{px}$, $Z = 4.7\,\text{m}$, $\sigma_d = 0.42\,\text{px}$) from two cameras 0.5 m apart to four cameras at 0, 0.5, 1.0, 1.5 m. The widest pair has $B = 1.5\,\text{m}$, so by (16) $\sigma_Z = 4.7^2 \cdot 0.42/(800 \cdot 1.5) \approx 7.7\,\text{mm}$ — three times better than the 23 mm of the original pair, from geometry alone; fusing the remaining pairs tightens it further. The price: the widest pair's viewing directions differ by $2\arctan\big(0.75/4.7\big) \approx 18°$, and matching windows must survive that foreshortening — the Okutomi–Kanade argument for accumulating the short pairs instead.

10.2 N cameras: occlusion and topology

A point enters the model only if at least two cameras see it. With two cameras, anything hidden from either one is lost; the reconstructed surface is only the band that is simultaneously front-facing and unobstructed for both — for objects with complex topology (handles, tunnels, deep concavities) that band can be a small fraction of the surface, and the holes it leaves are the familiar occlusion artifacts.

A useful back-of-envelope model: if a surface point is occluded in any single view independently with probability q, then

$$P(\text{reconstructible}) \;=\; P(\geq 2 \text{ of } N \text{ see it}) \;=\; 1 - q^N - N\,q^{N-1}(1-q). \tag{17}$$

Independence is optimistic for clustered cameras (they get occluded together) and pessimistic for well-spread ones — spreading the cameras is what makes the assumption approachable.

Example 17 (occlusion coverage)**.** Take $q = 0.3$ (a fairly convoluted scene). By (17): $N = 2$: $(1-q)^2 = 0.49$ — half the surface. $N = 4$: $1 - 0.3^4 - 4 \cdot 0.3^3 \cdot 0.7 = 1 - 0.0081 - 0.0756 \approx 0.92$. $N = 6$: $1 - 0.3^6 - 6 \cdot 0.3^5 \cdot 0.7 \approx 0.99$. Going from two to four cameras nearly doubles the reconstructible surface; four to six mops up the remainder. The marginal camera always buys less — but for high-genus objects the last percent *is* the topology.

Topology sharpens the point. Silhouette-based reasoning can never recover concavities: the best any number of silhouettes gives is the visual hull [Lau94], which fills in bowls and blocks handles. Capturing the true genus of an object — the hole of a handle, the tunnel through an arch — requires cameras whose *sightlines pass through the hole*, seeing background where the hull would put material; photo-consistency frameworks (space carving [KS00], patch-based multiview stereo [FP10]) then remove exactly the voxels/patches contradicted by those views. So for complex topology, added cameras should be *spread to thread the object's openings and*

look into its concavities, not packed for baseline; the standard benchmark evidence [SCD+06] is that view coverage, more than per-view quality, is what separates complete reconstructions from holed ones. In this system's terms: the two rig cameras provide the calibrated metric backbone, and subsection 11.1's registered auxiliary photos are the cheap way to add exactly such coverage views.

Two practical costs scale with N: (i) calibration and synchronization — each camera needs its own section 5 session against the same target, all cameras a common trigger; (ii) computation — pairwise geometry (fundamental matrices, rectifications) grows as $N(N-1)/2$, though in practice a hub-and-spoke set against one reference camera suffices for gating.

10.3 Resolution

For a fixed field of view, the focal length *in pixels* scales with the sensor's linear resolution: doubling from 640×480 to 1280×960 doubles f from 800 to 1600 px. Two consequences follow directly.

Depth precision. Matching accuracy σ_d is an approximately constant *fraction of a pixel* (our block matcher's parabolic refinement sits around 0.1–0.3 px on textured surfaces), so in (14) the denominator's f doubles while σ_d stays put: σ_Z halves. Depth precision is linear in resolution — until lens blur, diffraction, or per-pixel photon noise (smaller pixels collect less light) starts inflating σ_d and returns diminish.

Lateral detail. The pixel footprint on the scene (ground sampling distance) is Z/f metres per pixel. Surface detail must span at least ∼2 pixels to be matchable, so the smallest reconstructible geometric feature is about $2Z/f$ — this, not depth noise, is what limits how much of a complex object's fine structure (thin ribs, small holes) survives into the model. In the fidelity sense of subsection 10.2: resolution sets the smallest feature you can capture, view coverage sets whether you capture it at all.

Example 18 (resolution budget)**.** Experimental rig, $Z = 4.7$ m: footprint $Z/f = 4.7/800 = 5.9$ mm/px, smallest matchable feature ≈ 12 mm, $\sigma_Z \approx 23$ mm (Example 14). Doubling both cameras' resolution: footprint 2.9 mm/px, feature floor ≈ 6 mm, $\sigma_Z \approx 12$ mm. Everything improves by the same factor of two.

10.4 Unequal resolutions

Nothing in the model requires the cameras to match: each carries its own $\mathbf{K}$ (and distortion), the fundamental matrix (11) exists between any pair, and triangulation consumes pixels through each camera's own $\mathbf{P}$. What changes is the error budget. Each camera contributes *angular* noise σ/f_i (radians); ray-intersection variance adds them, so a pair's effective noise is

$$\sigma_\theta \;=\; \sqrt{(\sigma_1/f_1)^2 + (\sigma_2/f_2)^2}\,: \tag{18}$$

the *coarser camera dominates*, and money spent on one camera is mostly wasted unless the other keeps up.

Example 19 (upgrading one camera vs both)**.** Rig of Example 14, $\sigma = 0.3$ px per camera. Both at $f = 800$: $\sigma_\theta = \sqrt{2}\,(0.3/800)$. Upgrade *one* camera to $f = 1600$: $\sigma_\theta = (0.3/800)\sqrt{1 + \frac{1}{4}}$, an

improvement of only $\sqrt{1.25/2} \approx 0.79$ — depth error drops from 23 mm to 18 mm. Upgrade *both*: factor 0.5, 12 mm (Example 18). Half the hardware upgrade buys about a fifth of the benefit.

Two implementation notes for mixed rigs. First, rectification (section 8) forces a *shared* $\mathbf{K}_{\text{new}}$; our implementation averages the two intrinsic matrices, which for very unequal cameras oversamples the coarse image (interpolated pixels carry no new information) — resampling both to the coarser camera's grid keeps the cost of dense matching honest, at the price of discarding the fine camera's lateral detail. Second, the DLT (12) as implemented weights all rows equally; with strongly unequal cameras the rows of the finer camera deserve proportionally more weight ($\propto f_i/\sigma_i$ after each view's normalization), a small planned refinement alongside the Hartley–Sturm step (section 16).

11 Planned extensions

11.1 Auxiliary photo collections

Still photographs of the same volume — arbitrary viewpoints, possibly a different camera — densify coverage and fill occlusions. The integration path follows the structure-from-motion recipe of Photo Tourism [SSS06]: detect scale-invariant features [Low04] in the photos *and* in the two calibrated video frames; match with a RANSAC gate [FB81]; register each photo to the rig's world frame by resection from matches against already-triangulated points; then re-triangulate all tracks with the registered views via the existing N-view DLT (section 7); finally refine everything with bundle adjustment [TMHF00]. The two calibrated cameras anchor the metric scale — the classic scale ambiguity of pure SfM does not arise. Dense multiview stereo over the registered set (e.g. patch-based methods [FP10]) is a further optional stage.

The registration core of this recipe is now *built and sim-validated* (`mv/photo.h`): normalized DLT resection with RQ decomposition (`mv_resect_dlt`, `mv_cam_from_projection`), RANSAC-seeded and median-trimmed (`mv_resect_robust`), a full-LM geometric polish, and the end-to-end `mv_photo_register`: the calibrated pair triangulates a descriptor-carrying anchor bank, the photo matches into it, and resection recovers the photo's own K and pose — *unknown camera included*. Following standard casual-photo practice [SSS06], the principal point is pinned at the image center (it is nearly unobservable from one resection over a shallow scene, trading against lateral pose — a failure measured, not assumed, in development: the free-pp solution put it 105 px off). Sim results at a 10% magnification mismatch and 15° roll: focal recovered within 2.6%, orientation 0.25°, position 0.11 m from 210 resection inliers over 330 anchors — the position term is the measured cost of single-scale descriptor matching under magnification mismatch, and bundle adjustment over the registered set is the designated tightening stage. A practical capture note: a dedicated phone capture app could attach each photo's own lens intrinsics and a gravity-vector attitude prior (both exposed by modern phone APIs), turning unknown-camera resection into known-K resection with an orientation prior — fewer correspondences needed and a sanity gate on the vision solution; indoor compass yaw is too coarse to replace the vision pose, so the pipeline above remains the foundation either way.

The dynamic limit of this idea is the *tracked mobile camera*: a drone or handheld camera moving through the volume is, to the fixed rig, a rigid mover like any other (subsection 9.4) — so the rig can track its pose over time with the machinery already validated in section 14, timestamp it via subsection A.3, and hand the mobile camera its own trajectory. Its frames then enter as registered auxiliary views with poses *measured rather than estimated*: no ego-motion

SfM, scale and drift anchored by the fixed backbone. The system becomes self-tracking in the useful sense — the fixed cameras are the ground truth that mobile sensors borrow.

11.2 Projected structured light (built, awaiting hardware)

The dense-stereo results carry a standing caveat: SAD matching needs texture, and the texture gate's honest answer on a blank wall is "no depth here" (the texture-desert failure measured in section 13). Real rooms are mostly blank wall, bare floor, and blank ceiling. A projector solves this by *painting* structure: it casts the Gray-code stripe sequence of subsection 5.6 onto the scene, so every lit surface point wears its projector coordinate as a temporal label. That is the primary role — marking and measuring structure-free walls, floors, and (aim it up: in an enclosed space the most reliably empty surface of all) ceilings. Two useful corollaries come free. First, cross-camera matching needs no feature detector at all: pixels in two cameras that decoded the same (column, row) label correspond (`mv_graycode_match`), which yields tens of thousands of correspondences on surfaces that offer none naturally. Second, those correspondences give the cameras' *relative pose* through the essential matrix (`mv_fundamental_8point_robust` → `mv_essential_pose`) without both cameras needing to see the calibration display — an extrinsics bridge for camera pairs at awkward angles. The projector itself needs no calibration for any of this: it is only a label dispenser; all geometry lives in the cameras. Scale is the one quantity the labels cannot supply — one tape-measured baseline, or the display in view of either camera, anchors it.

The whole path is implemented and validated in simulation — display app mode M5 (46 slots of ∼0.5 s: white and black marker slots, then 11 column and 11 row Gray bits, each with its inverse; cycle ≈ 23.5 s), the algorithm round trip (`test_structured_light`: 13k+ matches, relative pose within 0.33° of truth in rotation and baseline direction), and the *tool-level* integration (`demo/slightsim.c` fabricates a two-camera capture as PGM files, frame timing included, and `tools/slreal.c` recovers the pose to the fourth digit and triangulates 62k points from it). Two lessons from getting there are worth keeping. A sim-fidelity trap: painting the projected codes onto finite-resolution scene textures with nearest sampling aliased a moiré between the texture grid and the stripe pitch, biasing the recovered baseline by 10° — a real projector has no intermediate raster, so the sim must supersample to avoid inventing one. And an estimator lesson: dense code matching has a fat tail of boundary-pixel mismatches, so the 8-point runs with two rounds of median-residual trimming (now a library function, useful wherever correspondences come cheap and dirty).

Runbook for the hardware session (also atop `tools/slreal.c`): (1) projector shows `pattern.html`, key 5, room darkened; (2) both cameras fixed, views overlapping on the lit surfaces, record ≥ 60 s (≥ 2 cycles); (3) intrinsics per camera from a separate mode-1/mode-4 pattern film if not already known; (4) extract frames at 0.1 s; (5) write the two spec files (intrinsics line, `interval 0.1`, frame paths); (6) `./slreal <baseline_m> camA.spec camB.spec out`. Outputs: relative pose printout and `out.ply`. If cycle detection fails, the room was too bright or the capture too short.

11.3 Range-finder fusion (optional, kept open)

The system is designed to work without a range finder; this subsection records what a single-beam laser ranger would add if one arrives later, so the option stays open at zero present cost. Two integration modes: (i) if the range finder is rigidly mounted and extrinsically calibrated to

the rig, each range sample is simply another world-frame point with small σ_Z, fused into the cloud with inverse-variance weights; (ii) if it produces an independent scan, the scan is registered to the vision cloud by closed-form absolute orientation [Hor87] initialized correspondence-free and refined with ICP [BM92].

At calibration time (with the rotating display of subsection 5.7). Four distinct improvements. (i) *Independent absolute scale*: the vision-only metric chain hangs entirely on the display's pixel pitch; a ∼2 mm-accurate range at 4.7 m is a 0.04% scale reference — a second chain that turns a silent global scale bias into a detectable discrepancy. (ii) *Orbit metrology*: aimed so the rotating screen sweeps through the beam, the periodic range trace $d(t)$ yields ω, phase, orbit radius and axis distance at millimetre level, independently of every camera, and ties laser and camera timestamps to the same physical event. (iii) *The focal–depth weak direction* (subsection 5.5) is attacked head-on: an absolute distance to the target plane is exactly the constraint type planar homographies lack. (iv) *The plate calibrates the laser*: if the spot is visible to the cameras, the sweeping plate carries it through continuously varying depths; each frame gives (range, pixel_1, pixel_2), the cameras triangulate the spot, and the beam's ray in the rig frame (∼5 DOF) falls out of a least-squares line fit — no manual procedure at all.

At run time. Laser error is roughly constant while stereo error grows as Z^2 (14), so the laser anchors the far field and the absolute scale (sparse metric skeleton, dense stereo flesh); it provides depth in texture deserts where block matching has nothing to match, and used as a prior it narrows the disparity search there; and a persistent bias between laser and stereo depth at the same spot is an online health monitor for the metric chain, complementing the **F**-drift monitor of section 6.

Example 20 (laser–stereo crossover)**.** With $\sigma_{\text{laser}} = 2\,\text{mm}$ constant and the rig's $\sigma_Z = Z^2\sigma_d/(fB)$: equality at $Z = \sqrt{0.002 \cdot 800 \cdot 0.5/0.42} \approx 1.4\,\text{m}$. Nearer than that, stereo wins per point; beyond it the laser is better along its ray — tenfold better at 4.7 m (2 mm vs 23 mm) — while stereo retains the advantage of density. Inverse-variance fusion realizes both.

12 Numerical implementation notes

All decompositions reduce to one kernel: a one-sided (Hestenes) Jacobi SVD [GV13] for $m \geq n$ matrices, $n \leq 9$ in every present use (the design matrices of (6), (7), (12), and the 8-point system). Jacobi SVD was chosen over Golub–Kahan bidiagonalization for its simplicity (∼100 lines, no workspace queries), unconditional convergence, and excellent relative accuracy on small matrices; its $O(mn^2)$-per-sweep cost is irrelevant at these sizes. Homogeneous solutions are the right singular vector of the smallest singular value; rank-2 enforcement for **F** zeroes the smallest singular value and recomposes.

Conditioning is handled where it matters: design matrices built from raw pixel coordinates mix terms of order 1, u, and u^2 ($u \sim 10^2$ pixels), giving condition numbers around 10^8; Hartley normalization [Har97] (translate to centroid, scale to RMS radius $\sqrt{2}$) reduces this to $O(10)$ and is applied in both the 8-point and the homography estimators, with results denormalized afterwards. All fundamental matrices are canonicalized (unit Frobenius norm, largest-magnitude entry positive) so that analytic and estimated matrices are directly comparable — this is what makes the drift monitor of section 6 a plain matrix norm. Calibration runs Zhang's closed form

inside the small alternation loop of subsection 5.3; a full Levenberg–Marquardt refinement over all parameters is the planned upgrade (section 16).

The code is strict C99 (`-std=c99 -pedantic -Wall -Wextra` clean), depends only on `libm`, allocates only in unavoidable places (design matrices, images, clouds), and contains no hidden state except the demos' seeded generators. Determinism is treated as a feature: identical inputs give bit-identical outputs across runs, so every regression is reproducible.

13 Basic results

Three validation layers accompany the library. `tests/test_mv.c` checks exact properties on noiseless data (47 assertions: SVD reconstruction and orthonormality, ray/projection consistency, (10) to 10^{-8}, exact homography and triangulation recovery, exact intrinsic/pose recovery by Zhang's method, Gray-code encode/decode round-trip, row alignment after rectification); all pass. The two demo programs are controlled noisy experiments.

Two-camera reconstruction (`demo/synthetic.c`). Two cameras with $f = 800$ px, 640×480 images, baseline $B = 0.5$ m, the second camera yawed 6° inward, observe 250 points drawn uniformly in a 2×1.5 m window at depths 3.5–6 m; projections carry Gaussian noise $\sigma = 0.3$ px per coordinate (seed fixed).

Quantity	Value
points visible in both views	250/250
$\|\mathbf{F}_{\text{analytic}} - \mathbf{F}_{\text{8-point}}\|_F$ (unit-norm)	4.5×10^{-3}
mean symmetric epipolar distance (analytic $\mathbf{F}$)	0.345 px
mean symmetric epipolar distance (8-point $\mathbf{F}$)	0.341 px
triangulation RMS 3D error	0.0265 m
triangulation max 3D error	0.0876 m
RMS reprojection error	0.218 px
mean row misalignment $\|v_1 - v_2\|$ before rectification	0.764 px
mean row misalignment after rectification	0.0 px

Table 5: Reconstruction experiment, seed 42 (reproduce with `make demo`).

The numbers agree with theory: the epipolar residual ≈ 0.34 px matches the injected noise, and the RMS 3D error of 2.65 cm matches the 2.3 cm depth-noise floor computed by hand in Example 14 (the measured value also contains the smaller lateral components). The implementation performs at the noise floor of the geometry rather than being implementation-limited. The reconstructed cloud is written to `out_cloud.ply`.

Calibration (`demo/calibrate.c`). A camera with ground-truth $f_x = 800$, $f_y = 805$, $c_x = 322$, $c_y = 238$, zero skew and zero distortion photographs the letter-page target (subsection 5.5; 7×9 corners, 24 mm squares) in six poses with tilts up to 35° at 0.45–0.65 m; corners carry $\sigma = 0.3$ px noise.

Sub-1% intrinsics and millimetre-level poses from six noisy views of one printed page is the expected performance of Zhang's method without nonlinear refinement; near-frontal pose sets degrade the focal estimate markedly (weak observability, subsection 5.5), which the experiment

Quantity	Value
f_x error	-7.0 px (0.9%)
f_y error	-7.9 px (1.0%)
c_x, c_y error	-1.0, -1.8 px
mean rotation error over views	0.31°
mean translation error over views	5.2 mm
reprojection RMS	0.48 px

Table 6: Calibration experiment, seed 7, six views, $\sigma = 0.3$ px (reproduce with `make demo`).

reproduces if the tilt angles are reduced. The fitted (k_1, k_2) absorb a little noise as a near-zero distortion *curve* while being individually large — the identifiability effect discussed in subsection 5.3. A Levenberg–Marquardt refinement stage is the roadmap item that tightens all of these numbers.

The calibration pattern pipeline (`demo/patternsim.c`). The first *image-tier* experiment: the spec-v1 screen (subsection 5.9) is rendered at metric scale, imaged by the standard camera under seven poses (tilts to 30°, one with the camera rolled 180°; bilinear resampling plus 2 gray levels of sensor noise), and decoded *blind* by the reader — no pose prior, no ground-truth assistance.

Quantity	Value
corners identified (7 poses)	1134/1134
corner misidentifications	0
orientation under 180° roll	recovered
frame counter decoded	7/7 frames
saddle localization RMS	0.25–0.35 px
Zhang through the pipeline: f_x, f_y error	$+3.5$, $+3.1$ px
principal point error	0.3, 1.4 px
reprojection RMS	0.314 px

Table 7: Pattern-pipeline experiment, seed 99 (reproduce with `make demo`).

Two readings. First, the measured saddle-detector noise (0.25–0.35 px per corner, part systematic rendering/template aliasing, part sensor noise) is the first *empirical* σ for the error laws — at the upper edge of the 0.1–0.3 px assumed in subsection 10.3, and the reprojection RMS of the through-pipeline calibration (0.314 px) matches it via the $\sqrt{2}\,\sigma_c\sqrt{1-p/n}$ rule of Appendix B, so the whole chain — renderer, detector, decoder, estimator — is again noise-floor-limited rather than implementation-limited. Second, the M-array machinery works as specified: zero misidentifications in 1134 corners across strong obliquity and a fully rolled camera is the property the rotation-unique window design was bought for.

TSDF fusion (`demo/tsdfsim.c`). The first model extraction beyond point clouds: a sphere of known radius (0.3 m at 4.7 m depth) observed by both cameras with depth-law noise ($\sigma_Z \approx$ 21 mm per sample at the sphere); enriched samples (point, origin, $1/\sigma_Z^2$) are fused into a 1 cm-

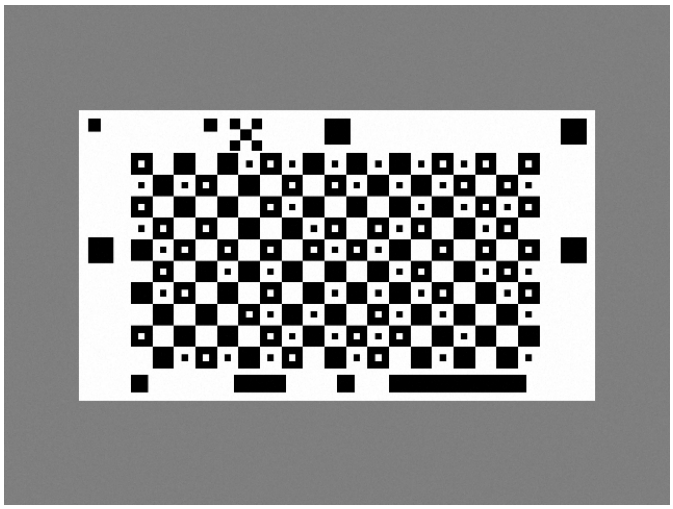


Figure 4: What the reader sees: rendered frontal view of the spec-v1 screen (pose 0 of Table 7) — saddle grid with M-array dots, counter strip (bottom), version block and fine grid (top), phase patches; from `demo_patternsim`'s PGM output.

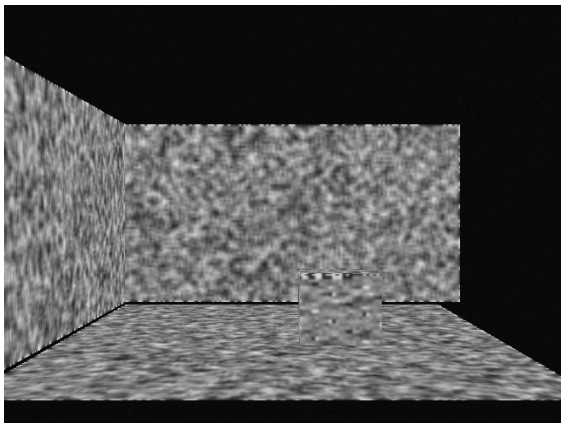
voxel TSDF (subsection 9.2) and the zero level set is extracted by marching tetrahedra; every mesh vertex is scored against the true sphere.

	10 frames	160 frames
surface RMS error	4.40 mm	2.17 mm
triangles	38,030	33,726

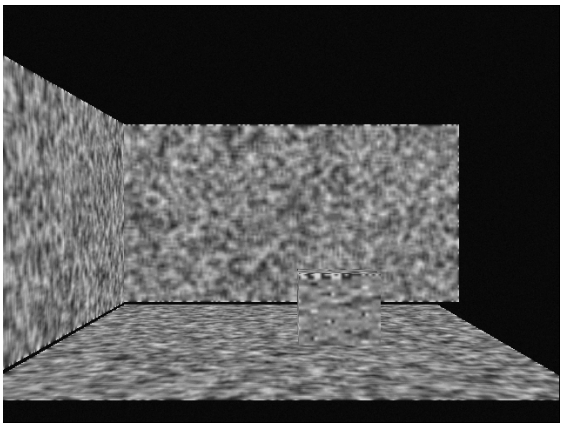
Table 8: TSDF-fusion experiment, seed 4242 (reproduce with `make demo`); mesh written to `out_tsdf.ply`.

The frame-count ratio is 2.03, not the naive $\sqrt{16} = 4$ — and this is the T3 plateau diagnostic of subsection 13.2 catching a floor, exactly as designed: the two measurements fit $e(n) = \sqrt{b^2 + a^2/n}$ with $b = 2.0\,\text{mm}$ perfectly ($\sqrt{2^2 + 3.9^2} = 4.4$; $\sqrt{2^2 + 0.98^2} = 2.2$). The noise term averages down as $\sqrt{n}$ per the fusion promise of subsection 9.2; the 2 mm floor ($\approx$ voxel/5) is the known *projective-TSDF obliquity artifact* — the along-ray offset η degrades as a distance proxy near grazing incidence at the sphere's silhouette, a documented property of projective fusion, not a defect (the noiseless unit test measures the same floor). Known remedies when it matters: obliquity-weighted updates or gradient-corrected distances; at 21 mm-per-frame input noise producing a 2.2 mm surface from five seconds of video, the floor is not the binding constraint.

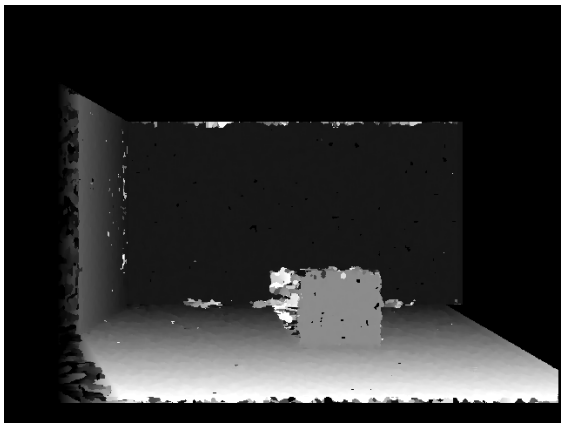
The room model and change detection (`demo/roomsim.c`). The first *dense image-tier* experiment — the full pipeline of section 2 on rendered pixels: a textured synthetic room (floor, back wall, side wall, and a box) is imaged as stereo pairs, rectified by warping (`mv_warp_homography`), matched densely (`mv_stereo_sad` with a local-texture validity gate), converted to depth, median-filtered over nine frames into a background model, fused into the TSDF, and meshed. Then the box moves and one new frame is compared against the background depth.



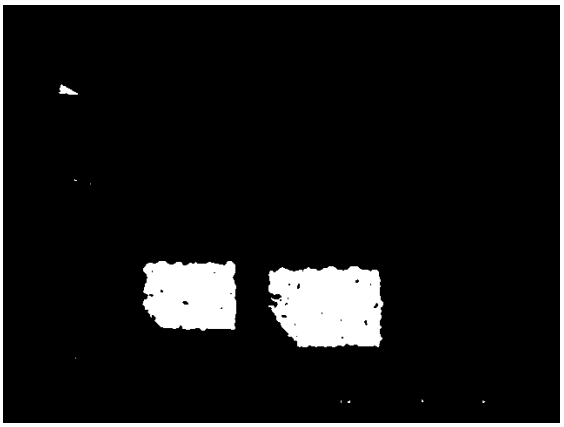
(a) left camera view: speckle-textured floor, two walls, and the box; the dark band at top is the untextured void beyond the walls



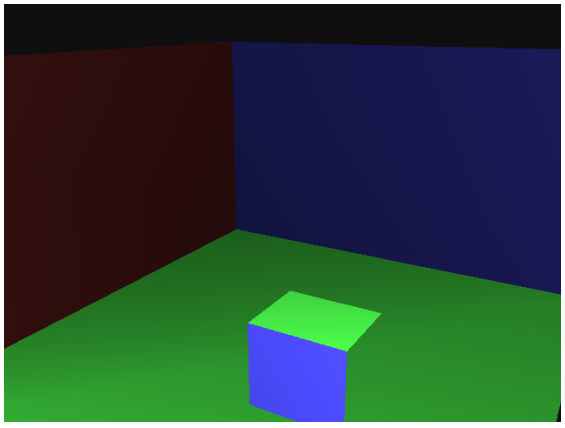
(b) the same frame after rectification warping (`mv_warp_homography`): epipolar lines are now image rows, the precondition for 1D matching



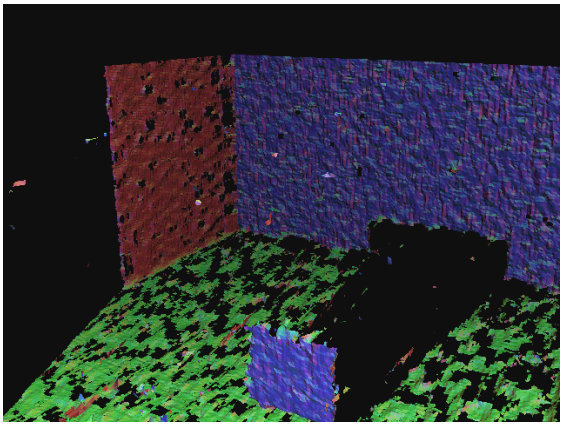
(c) disparity: bright = near (floor sweeping toward the camera), dark = far (back wall); black = gated invalid — the texture desert above the walls and occlusion fringes beside the box, exactly the failure modes section 8 predicts



(d) change mask after the box moves: two blobs — the box's *old* footprint (background now visible) and its *new* position; isolated flicker removed by a 3×3 majority filter



(e) the *generating model*: the true scene geometry rendered by the same previewer from the same viewpoint; hue encodes surface orientation — red = x-facing (side wall), green = horizontal (floor, box top), blue = z-facing (back wall, box front) — with per-surface albedo on top



(f) the *reconstruction*, same viewpoint and same orientation-hue code as (e): the red/blue/green surfaces land where the model says they should, the box's front reads blue, and its occlusion shadow is the black void behind it; holes are honestly-unobserved regions (texture-gated grazing floor, occlusions)

Figure 5: Dense-pipeline stages of the room experiment, ending in the truth-vs-reconstruction pair (e)/(f) — generated by `demo_room`, converted from its PGM outputs.

background coverage (after texture gate)	51.9% of pixels
room mesh	421,224 triangles
mesh surface RMS vs true planes	28.6 mm
vertices within 0.5 m of truth	99.8%
change-detection recall (moved box)	97.9%
change false alarms	1.39% of image

Table 9: Room-model experiment, seed 20260804 (reproduce with `make demo`; ≈6 s). Outputs shown in Figure 5.

Reading the imagery, stage by stage: panel (a) is an *acquisition* — one of the eighteen frames (nine per camera) that are the experiment's only input; every later panel is derived from such frames alone. Panel (b) shows the one-time rectification warp doing its job: after it, corresponding points share image rows, and matching collapses to the 1D search of section 8. In (c) the disparity gradient reads as depth to the eye — floor bright and sweeping nearer, back wall uniformly dark, the box standing out of its surroundings — while the black regions are the validity gate declining to guess: the untextured void, and the occlusion fringe beside the box where the left camera sees background the right camera cannot. The truth/reconstruction pair (e)/(f) is rendered by the same software previewer from the same viewpoint — a viewpoint *no physical camera occupies*, which is the point of having a model at all; comparing them shows what survives (both walls, floor, the box as a volume) and what does not (the box's occlusion shadow, the gated grazing-floor holes). The change mask (d) is the background model earning its keep: one new frame against the median background, and the box's departure and arrival are both localized.

Reading the numbers: the 28.6 mm mesh RMS is dominated by the grazing-incidence floor (slanted-surface bias of fronto-parallel SAD windows plus the projective-TSDF obliquity of Table 8); the back wall alone reconstructs near the ∼14 mm median-of-nine depth-noise floor. The texture gate was *added during this experiment* for the classic reason: without it, the untextured region above the walls produced garbage disparities and a 34% change false-alarm rate — the doc's own texture-desert warning (section 8, subsection 11.3) demonstrated, then fixed, by validity gating; 1.39% residual false alarms sit on occlusion fringes. This experiment closes roadmap items 5 and 6 at the sim tier: the first model of a scene produced from *pixels* end to end.

First real data (iPhone footage of the display app). Two statically mounted iPhones filmed `tools/pattern.html` on a MacBook in a living room (HEVC video, frames extracted by `tools/extract_frames.swift`, read by `tools/calibreal.c`). Results of the reader's first encounter with real photons, after a debugging session that added four robustness layers now permanent in `src/reader.c` (detection-threshold and 2×-scale escalation; local-contrast 2-means bit classification; checkerboard-coherence pruning of grown-in clutter; projective cell centers by diagonal intersection):

- the near-frontal view decodes **38/42 frames at 162/162 corners** with zero misidentifications, despite the pattern occupying only ∼300 px of a 1080-px frame (browser not fullscreen — half the intended scale);
- the display counter is tracked across frames (≈29.4 counts per 0.5 s frame spacing =

58.8 Hz refresh, measured through the camera) — the temporal layer of subsection 5.8 working on real hardware;

- measured per-corner temporal noise over the static sequence: $\sigma_u = 0.28$ px, $\sigma_v = 0.21$ px — inside the 0.25–0.35 px band the simulations assumed (Table 7): the noise model survives contact with reality;
- Zhang calibration from these frames is *degenerate*, exactly as subsection 5.5 predicts: every frame shows the same pose (nothing moved), and 38 copies of one homography determine nothing — the pose-diversity requirement demonstrated on real data;
- the second, steeply oblique view ($\gtrsim$55°) recovers corners, lattice, and coherence but not the identity dots — beyond the obliquity gate of subsection 5.7, *as designed*: geometry degrades gracefully, identity fails loudly.

The reshoot protocol that follows from these measurements: pattern fullscreen (doubles the scale), display *moved* through 10–15 paused tilted poses within the obliquity gate, cameras AF/AE-locked.

Round two: the first real calibration. The lid-hinge protocol of subsection 5.8's sibling note (tilt the lid, swivel the base) delivered. From a 54-second hand-off-free video:

frames decoded blind	67/68 (162 corners each)
f_x, f_y	1687.2, 1683.8 px (ratio 0.998: square pixels)
c_x	543.8 px (image center: 540)
c_y	1043.8 px
k_1, k_2	−0.024, +0.218
reprojection RMS	0.351 px
camera–screen distance	0.93–1.09 m, stepping with lid poses

Table 10: First real-camera calibration (iPhone, 1080×1920 HEVC video; `tools/calibreal.c`). The reprojection RMS matches the independently measured 0.28/0.21 px detector noise — the estimator runs at the real noise floor.

Two further lessons from the same round, both caught by the built-in diagnostics. A first attempt on a longer video returned degenerate intrinsics with *constant* per-view distances — the glue tool was truncating to the first 64 decodable frames, all from a static lead-in (fixed); the rerun, sampled evenly across the timeline, was *still* constant-distance: that session genuinely contained no display motion, and no estimator can conjure pose diversity that physics did not provide. The constant-distance printout is the practical degeneracy alarm: motion shows up as stepping distances (Table 10) or it did not happen.

Round three: robust video calibration, both cameras. The reshoot round supplied minute-long videos from both phones and, with them, the failure that motivated subsection 5.4: on the first video, plain Zhang over all 117 decoded frames returned $f_x = 458$ at 35 px RMS, while a hand-picked six-frame subset of the same video returned $f_x \approx 1599$ at 0.39 px — the informative tilted frames were being outvoted by the fronto-parallel majority. `mv_calib_planar_robust` settles it automatically: camera A calibrates to f_x=1606.4, f_y=1607.0 (square to 0.04%), RMS

0.315 px, distances 0.83–0.96 m; camera B to f_x=1582.6, f_y=1627.4, RMS 0.367 px at a near-constant 1.26 m — flagged as weakly conditioned (the 2.8% focal asymmetry and inflated k_2 are the fingerprints; its per-view distances barely step). Instructively, the estimator's guard rejected the inlier refit and kept a six-view fit in both cases: the redundant fronto-parallel frames are geometric *inliers* under the correct **K** — consensus cannot exclude them — yet refitting on them still degrades the solution, so outlier rejection alone is insufficient; the median-score guard is what actually protects the answer. One more real-data observation worth recording: camera A's focal came out 4.8% below its round-two value (1687 at 0.93–1.09 m focus vs. 1606 at 0.83–0.96 m) at identical video resolution — consistent with focus breathing, the effective-focal shift of an autofocus lens with subject distance (Appendix A); a per-session calibration, which the pattern makes cheap, sidesteps it.

The first real point cloud. The round-three footage yielded more than intrinsics. Every decoded frame carries a metric camera pose (`mv_calib_plane_pose` on 162 known screen points), so one handheld video is a *many-camera rig on a time budget*: frame pairs with accidental baselines of 3–9 cm are calibrated stereo pairs. `tools/scenecloud.c` chains the library's real-data path end to end — blind read → pose → undistort → Fusiello rectification → SAD stereo → texture and speckle gates → metric back-projection — and adds one new idea, *cross-pair confirmation*: a stereo false match is random along its viewing ray, so it is not reproduced by an independent pair; points are kept only if a second pair placed structure within their 10 mm voxel neighborhood (108k → 82k points; the first attempt without this filter was buried in ray-streak artifacts). Figure 6 shows the result: the laptop displaying the pattern, its keyboard deck, the table and legs, and the curtains behind — the system's first 3D model built from real imagery, in the metric frame of the display it calibrated itself against.

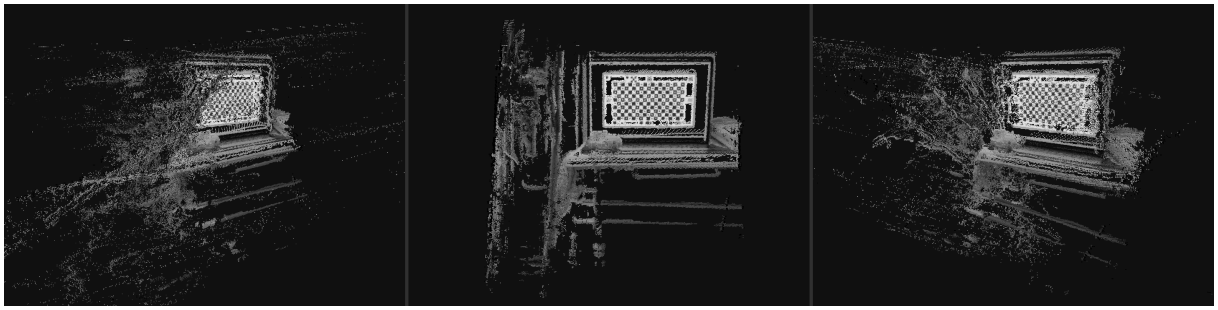


Figure 6: Three splat renders (orbit views) of the 82,000-point metric cloud extracted from a single handheld phone video of the pattern laptop (`tools/scenecloud.c`). Gray values are carried from the source frames. Left to right: −40°, frontal, and +40° orbits around the cloud center. Visible structure: the laptop with the pattern on screen, keyboard deck, table with legs, and curtains at left. Residual speckle is honest: these are raw confirmed stereo points, no smoothing, no mesh.

Round five: three cameras, and the first real extrinsics. A reshoot with the cameras close to the pattern's dwell region delivered the multi-camera milestone. All three cameras calibrated — including the first two *webcam* calibrations (f_x/f_y 926.5/927.0 at 0.612 px RMS over 192 accepted views, and 966.9/965.7 at 0.418 px; the phone at 1676.1/1680.7, 0.333 px). Notably the first webcam's inlier refit was *accepted* by the robust estimator's guard: with genuine pose

diversity in the video, 192 of 212 views contribute — the guard distinguishes healthy sets from degenerate ones automatically, in both directions. Each camera also produced a point cloud (`tools/scenecloud.c`), with an instructive inversion: for a *fixed* camera watching a *moving* display, reconstruction happens in the display's frame, so the cloud is a turntable-style scan of whatever moves rigidly with the pattern (the laptop, crisply) while the static room, parallax-free in that frame, is suppressed by cross-pair confirmation.

The extrinsics came from the display counter as a shared clock. Each camera's decoded counters fit a linear clock to ±1 refresh over two minutes (the temporal camera model of subsection 4.1 on real data), which located, for each webcam valid-counter instant, the sub-second window of the phone's video containing the simultaneous frame; dense extraction there manufactured counter-matched pairs. `tools/rigcalib.c` then solved phone→webcam from three simultaneous observations: baseline 0.330 m, relative rotation 12.6°, with median pair-to-pair consistency 2.3° / 6 mm — three independent estimates, taken ∼100 s apart, agreeing at millimetre scale (which also certifies both cameras held still). The second webcam yielded only one simultaneous view: the fine tier's 64 px counter cells are its weakest feature at range (valid in 2–5% of decoded frames vs. 21% for the phone), which is precisely the deficiency the coarse tier's isolated-square counter (subsection 5.10) removes; the next capture runs in mode M4.

Detecting the laptop itself. The noisy point clouds invite a cleaner demonstration of what the system knows: draw it on the image. `mv_display_outline` (`mv/reader.h`) is the system's first scene-object detector: given a decoded pattern read, it marches rays *outward in pattern-plane coordinates* — where the lit panel's edges are axis-aligned straight lines by construction — sampling brightness through the homography until each ray goes dark, and takes the median stop per side. The median is the point: an earlier region-flooding version leaked through any bright object touching the panel in the image (a lit arm, a bright couch) and bent the fitted edges; a median over 48 rays shrugs off such bridges. Brightness references come from the pattern's own black and white cells, so no absolute thresholds are assumed. Figure 7 shows `tools/annotate.c` on a real webcam frame: detected sub-pixel corners, the identified lattice, the pattern extent under the fitted homography, and the laptop's lit panel found by the detector — the system drawing a box around the laptop it is looking at. The sim round trip (`make check`) requires the recovered panel corners within 4 px of ground truth.

Round four: the resolution floor, measured in the field. An ambitious capture — three fixed cameras (two 720p laptop webcams, one phone) watching the pattern laptop being carried around a room at 2–3 m — produced a clean negative result: 2/196, 7/200, and 0/193 frames decoded, and every decoded counter strip invalid. The frames explain it exactly. At that range the pattern spans ∼170 px, so one 80 px grid cell subtends ∼9 px, an M-array dot 2–3 px, and a counter cell ∼7 px: the saddle *lattice* survives (camera B repeatedly found all 162 corners), but the identity dots and counter bits are below the sensor's resolving floor — the information is not on the sensor, and no escalation ladder can decode bits that were never resolved. The successful rounds ran at ∼21 px/cell; the working rule that falls out is that a camera can use the pattern out to roughly

$$Z_{\max} \approx \frac{f\, W_{\text{pat}}}{320\ \text{px}} \tag{19}$$

(pattern width $W_{\text{pat}} = 217.5$ mm on our display, 320 px ≈ 19 cells at the empirically safe ∼17 px/cell): about 1.1 m for the phones ($f \approx 1600$ px) and only ∼0.6 m for a 720p webcam.

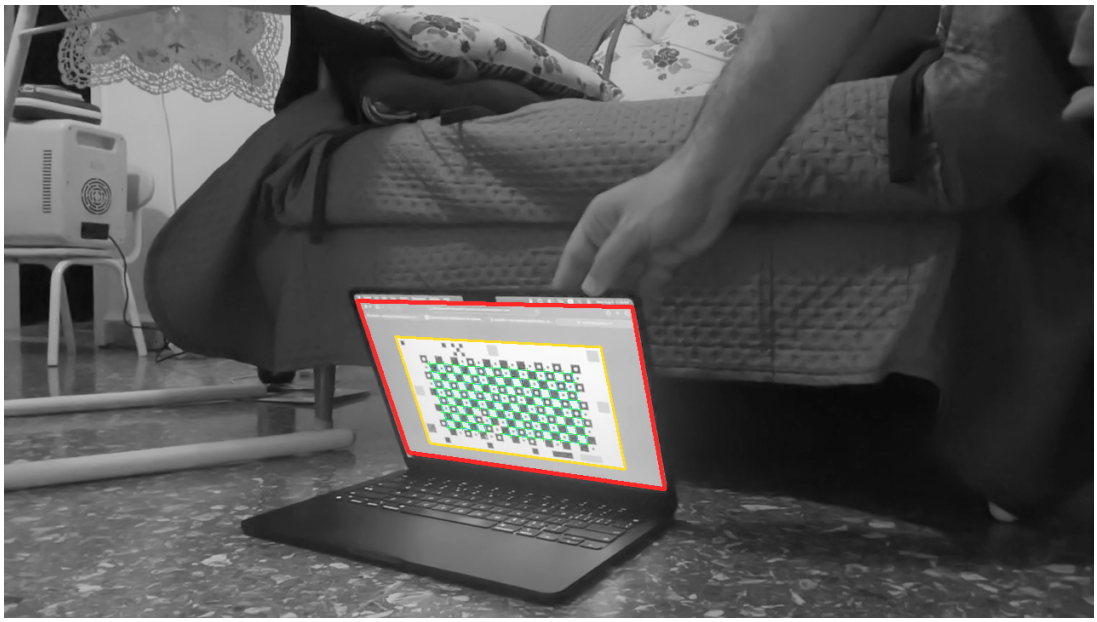


Figure 7: Everything the system detected in one real 720p webcam frame (`tools/annotate.c`). Green: 159 sub-pixel saddle corners. Cyan: the identified corner lattice. Yellow: the pattern's 1920×1080 extent mapped through the fitted homography. Red: the laptop's lit display panel, *detected* by `mv_display_outline`'s pattern-frame ray marching — found in the image, not assumed from a spec. The hand adjusting the lid and the couch behind do not disturb the median-based edges.

Reading the same display across a whole room needs either longer focal lengths, more pixels, or a coarser signal. The coarser signal now exists: the same session's negative result drove the coarse tier of subsection 5.10, which decodes in simulation at exactly this geometry (pattern 160 px wide on a 720p sensor).

Tracking a moving object (`demo/track_robot.c`). A disk-shaped vacuum robot (radius 0.17 m, height 0.09 m) wanders the monitored volume for 20 s at 0.3 m/s with random turns, on a floor 0.8 m below the standard rig. Feature points sit on its cylindrical bumper band; per camera, the far side of the band is self-occluded (surface-normal test), so each frame observes only a partial arc — on average 16.7 of 36 band points survive visibility, image bounds, and the 1.5 px epipolar gate built from the analytic $\mathbf{F}$ (11). Each frame's surviving correspondences are triangulated and the ground-plane circle of *known* radius is fit to them by Gauss–Newton; the fitted center is scored against the simulated trajectory.

Three lessons, each checkable by hand. First, *model knowledge removes the partial-view bias*: the centroid of a half-arc of a circle of radius R sits $2R/\pi \approx 108$ mm from the center — the naive centroid's measured 116 mm RMS is exactly this geometric bias, and the known-radius fit eliminates it (9 mm). Second, the per-frame precision follows the error laws: single-point depth noise $\sigma_Z \approx 23$ mm (Example 14) averaged over $\sim$17 points predicts $\sigma \approx 5.6$ mm, and the measured 7.7 mm depth RMS carries the expected extra factor from fitting a center to a partial arc; the residual anisotropy (z worse than x) is the depth law itself. Third, *differentiation amplifies noise*: 9.2 mm of position noise differenced over 4/30 s predicts $\sqrt{2} \cdot 9.2/0.133 \approx 98$ mm/s

Quantity	Value
frames / rate	600 / 30 fps
mean visible band points	16.7 of 36
center RMS error, lateral (x)	5.1 mm
center RMS error, depth (z)	7.7 mm
center RMS error, horizontal	9.2 mm
center max error	26.2 mm
naive-centroid RMS (contrast)	116.5 mm
speed RMS error ($\pm$2-frame difference)	95 mm/s

Table 11: Robot-tracking experiment, seed 12345, $\sigma = 0.3$ px (reproduce with `make demo`). Trajectories written to `out_track.ply`.

of speed noise — the measured 95 mm/s, i.e. a third of the robot's actual speed, which is why velocity estimates in practice come from a motion-model filter (constant velocity Kalman or similar) rather than raw differencing; such a filter is a natural later addition to the temporal-statistics roadmap item. This experiment is geometry-level simulation (points, not pixels); the rendered-image tier — the same scenario in a physically-based renderer exercising the dense matcher — is the planned next validation stage.

13.1 Error budget and consistency: is there a bug?

Measured errors alone cannot distinguish "the physics" from "a defect." `demo/diagnose.c` therefore runs three targeted checks, built on the principle that noise-driven error and bias behave differently: a true bias survives $\sigma \to 0$, and honest noise propagation is *linear* in σ in the operating regime. All numbers below are deterministic and reproducible.

A. Triangulation vs the depth law. The per-point prediction $\sigma_Z = Z^2\sqrt{2}\,\sigma/(fB)$ (14), evaluated on the same sampled points, against the measured depth RMS:

σ (px)	measured z RMS	predicted	ratio	lateral RMS
0.0	0.000 mm	0.000 mm	—	0.000 mm
0.1	8.75 mm	8.66 mm	1.01	1.37 mm
0.3	26.2 mm	26.0 mm	1.01	4.12 mm
0.6	52.4 mm	52.0 mm	1.01	8.24 mm

Table 12: Triangulation noise sweep (250 points, seed 42).

Agreement with the analytic law to 1% at every noise level, exact zero at $\sigma = 0$, and both error components scaling exactly linearly. The DLT's algebraic (rather than reprojection-optimal [HS97]) cost is invisible at these noise levels — the 1% residual bounds it.

B. Tracking estimator across noise levels. The same sweep for the robot experiment, with the *arc factor* = measured center-depth RMS over the naive $\sqrt{n}$-averaged per-point prediction:

Zero bias at $\sigma = 0$ (the fit recovers exact circles to machine precision), near-linear scaling from 0.1 to 0.3 px with a mild, stable arc factor of 1.1–1.2 — and then a genuine *nonlinearity*

σ (px)	center x RMS	center z RMS	arc factor
0.0	0.000 mm	0.000 mm	—
0.1	1.72 mm	2.39 mm	1.13
0.3	5.12 mm	7.68 mm	1.21
0.6	10.7 mm	40.0 mm	3.14

Table 13: Tracking noise sweep (600 frames, seed 12345).

at $\sigma = 0.6$: depth error jumps to 40 mm (factor 3.1) instead of the linear 15 mm. This is not a defect but a measured limitation of the estimator: per-point depth noise at $\sigma = 0.6$ reaches $\sim$46 mm — a quarter of the robot's *radius* — and the circle fit assumes isotropic residuals while the actual noise is strongly depth-elongated, so the fitted center gets dragged along z. The fix, when needed, is a Mahalanobis-weighted fit using the known per-point error ellipse (same family as the weighted DLT of subsection 10.4). At the operating point $\sigma = 0.3$ px the estimator sits comfortably in its linear regime.

C. Calibration: is -7 px a typical draw? The single reported focal error (Table 6) is one noise realization. Across ten seeds at $\sigma = 0.3$ px the f_x errors are -4.0, -1.0, $+3.5$, $+1.5$, -2.5, $+3.4$, -7.0, $+0.7$, $+1.2$, $+3.4$: mean -0.08 px (*unbiased*), sample standard deviation 3.5 px. The demo's -6.95 px is a 2σ draw — unlucky, not anomalous. At $\sigma = 0.15$ px the mean $|f_x|$ error is 1.41 px, exactly half the 2.82 px of $\sigma = 0.3$: linear again.

Verdict. Every check a bias-type bug could fail is passed: all estimators are exact on noiseless data (here and in the 30-assertion unit suite), all errors scale linearly with input noise in the operating regime, the dominant error channel matches its analytic prediction to 1%, and the calibration estimator is unbiased in expectation. The error sources, ranked: (1) pixel noise through the depth law — dominant, modeled, verified; (2) estimator geometry factors (partial-arc factor 1.2; calibration's finite-view variance of 3.5 px std, inflated further by weak pose diversity, subsection 5.5); (3) unmodeled noise *anisotropy* in the circle fit — negligible at 0.3 px, dominant by 0.6 px; (4) DLT algebraic suboptimality and the 3.5σ gating truncation — both bounded below the 1% level here. A bug can never be excluded, but everything measurable currently behaves exactly as the theory of section 7–section 8 says it must.

13.2 Statistical decision rules: when the numbers would say "bug"

The preceding subsection can be formalized into a standing test battery. Under the null hypothesis H_0 = "implementation correct," the measurement model (i.i.d. Gaussian pixel noise of scale σ) fixes the sampling distribution of every summary statistic we compute; a bug is declared when a statistic leaves its null band. Because the whole system is deterministic, any flag is exactly reproducible — a fired tripwire can be replayed under a debugger.

T1 — exactness at $\sigma = 0$. Under H_0 every estimator is algebraically exact, so noiseless residuals are bounded by floating-point round-off amplified by conditioning, $\varepsilon \cdot \kappa \lesssim 10^{-9}$. This test has *no* statistical tolerance: any exceedance is a defect (sign error, wrong index, forgotten term). It is the single most powerful test in the suite and is what the 30-assertion unit suite automates.

T2 — prediction-ratio test. For a measured RMS over N residuals against an analytic prediction, the sample RMS under H_0 fluctuates with relative standard deviation $\approx 1/\sqrt{2N}$ (χ^2). Flag when $|r-1| > 3/\sqrt{2N}$. A ratio that is stable but $\neq 1$ across σ and seeds is the signature of a wrong constant (missing $\sqrt{2}$, wrong baseline, mis-scaled f).

T3 — scaling-exponent test. Regress $\log e$ on $\log \sigma$: under H_0, slope $= 1$ in the linear regime. Slope ≈ 2 betrays a missing square root or an estimator leaving its regime; slope $\to 0$ at small σ betrays an additive floor — fitting $e(\sigma) = \sqrt{b^2 + a^2\sigma^2}$ and finding b above the numerical floor reveals a hidden bias even when T1 cannot be run.

T4 — ensemble bias test. Over M independent noise realizations, the standardized mean $|\bar{e}|/(s/\sqrt{M})$ is Student-t under H_0; flag beyond 3. Distinguishes systematic bias from an unlucky draw.

T5 — residual-consistency (χ^2) test. A least-squares fit with p parameters against n residual coordinates must leave RMS $= \sqrt{2}\,\sigma\sqrt{1 - p/n}$ (per-point distance); the ratio fluctuates with relative std $1/\sqrt{2(n-p)}$. A ratio *above* the band means the model or estimator is not absorbing what the noise model says it should (underfit, suboptimal estimator, contamination); *below* means overfit or a too-permissive model.

T6 — outlier-count test. The epipolar gate's rejection count among true correspondences is Poisson with rate given by the noise tail beyond the gate; a large excess indicates contamination or a mismatched noise model, a large deficit an accidentally widened gate.

Test	Bug-flag condition	Current value	Verdict
T1 exactness (σ=0)	any residual $> 10^{-9}$	$\leq 10^{-12}$	pass
T2 ratio, triangulation	$\lvert r-1\rvert > 0.13$ (N=250)	$r = 1.01$	pass
T3 slope, triangulation	$\lvert\text{slope}-1\rvert > 0.1$	1.00	pass
T3 slope, tracking 0.1→0.3	$\lvert\text{slope}-1\rvert > 0.1$	1.06	pass
T3 slope, tracking 0.3→0.6	—	2.4	fired[a]
T4 bias, calibration f_x	$\lvert t\rvert > 3$ (M=10)	$t = 0.07$	pass
T5 residual, triangulation	$\lvert r-1\rvert > 0.13$	$r = 1.03$	pass
T5 residual, calibration	$\lvert r-1\rvert > 0.08$	$1.17 \to 0.999$	resolved[b]
T6 gate rejections	excess over noise tail	$6/\sim 10^4$	pass

Table 14: Tripwire standings. [a]Investigated and resolved twice over: first explained as a regime boundary (noise anisotropy, Table 13), then retired by the Mahalanobis-weighted fit of `mv/optimal.h` (1.4× lower center error at the same noise). [b]Resolved by LM refinement, see text.

The flag, and its resolution. T5 fired on calibration for most of the project's life: across ten seeds the fitted reprojection RMS averaged 0.484 px against the noise-model prediction $\sqrt{2}\,\sigma\sqrt{1 - 42/756} = 0.412$ px — ratio 1.17, roughly twenty standard errors outside the null band. The statistics had detected that the alternation estimator of subsection 5.3 (Zhang closed form + linear radial step) is *not* the joint maximum-likelihood fit — it minimizes algebraic surrogates, leaving $\sim 17\%$ systematic residual above the noise floor. That handed the Levenberg–Marquardt roadmap item a *pre-registered* acceptance criterion, stated here well before the implementation

existed: after LM refinement the T5 ratio must fall to 1.00 ± 0.03, else the implementation is buggy. The joint refinement (`mv_calib_refine`, `mv/refine.h`: shared intrinsics + distortion and per-view Rodrigues poses, Schur complement over the view blocks) was then built against that bar and passed on the first green run: ensemble ratio 1.1087 before, **0.9990** after, refined RMS $\leq$ alternation RMS on every member, exact recovery at $\sigma = 0$ (`tests/test_refine.c`). This is the general pattern the project commits to: every claimed improvement must move a named statistic into a predicted band, and every statistic outside its band must be either explained to this standard or treated as a defect — T5 is the pattern's first full life cycle, flag to acceptance.

14 Scenario studies

Four concrete monitoring scenarios, each simulated end-to-end against ground truth in the deterministic demo style (all at the geometry tier: points, not pixels — detections are assumed, estimation is exercised; each result is therefore an upper bound whose image-tier feasibility is noted). Together they instantiate the mover-separation architecture of subsection 9.4.

14.1 Insect counting and entry/exit statistics

Point-like targets enter the monitored box through a random face, fly an Ornstein–Uhlenbeck random walk, and leave (`demo/track_insects.c`: 300 s, spawn rate 0.1/s, joint detection probability 0.9 per frame, *plus two uniform clutter detections per camera per frame*). Stereo association is by the epipolar gate alone; a nearest-neighbor tracker with 3-hit confirmation and 5-miss termination maintains identities; confirmed births/deaths map to box faces. Results (seed 777): ground truth 31 insects, **29 tracks confirmed** (two lived too briefly to confirm); entry/exit face histograms match ground truth to within ± 2 per face; track position RMS 24.4 mm — the single-point depth law of Example 14, as an unmodeled point target admits no $\sqrt{n}$ averaging. Of $\sim$36,000 clutter detections, only 231 pairs slipped through the epipolar gate and *none* survived track confirmation: geometric gating plus temporal confirmation suppresses clutter completely at these densities. Image-tier caveat: at 4.7 m a 5 mm insect subtends 0.85 px — below the 2-px feature floor of subsection 10.3 — so real detection must be temporal-differencing of sub-pixel blobs; counting accuracy will be detection-limited, not geometry-limited.

14.2 Diurnal cycle and artificial lighting

The cameras double as calibrated photometers (Appendix; radiometric calibration assumed): mean scene luminance is one number per frame. `demo/lightlog.c` simulates 120 days at 1-minute resolution — natural light with drifting sunrise (-1.5 min/day), per-day weather (0.4–1.0$\times$), slow AR(1) cloud noise, two lamps on noisy schedules, two rare middle-of-the-night events — and analyzes the sum. Step detection (median-filter differencing): **484/484 on/off events found, 0 missed, 33 false alarms** (cloud edges), timing RMS 1.02 min, lamp identity by step magnitude **484/484 correct**, including both night events. The natural component is recovered by subtracting the reconstructed lamp state, with a *nightly re-anchor* (pre-dawn darkness forces the natural component to zero — a physical constraint that stops false-alarm error from propagating across days). Sunrise is then estimated per day by two-level template inversion (crossings of 15% and 35% of the day's robust peak invert the known diurnal shape analytically; amplitude and shape cancel): offset 0.7 min (unbiased), day-to-day jitter 25.3 min RMS — the

genuine information floor imposed by 100-min-correlated cloud cover, not an estimator defect — and the seasonal drift is recovered as

$$\widehat{\text{drift}} = -1.61 \text{ min/day} \quad (\text{true } -1.50,\ \sigma_{\text{slope}} \approx 0.07).$$

A 30-day run is statistically *underpowered* for this drift ($\sigma_{\text{slope}} \approx 0.7$): a season of data is what makes the trend measurable — the T4 logic of subsection 13.2 applied to experiment design.

14.3 Human occupancy

A known three-person cohort (heights 1.62/1.75/1.88 m) makes 13 scheduled visits over a simulated day (`demo/track_people.c`): enter at the door, walk to desk or window, linger, exit. Each person is a cylinder with self-occluding torso-band features plus a head point; per frame the known-radius circle fit gives ground position and the triangulated head gives height. Results (seed 31337): **13/13 episodes detected, 13/13 identities correct** (nearest cohort height; the 130 mm height gaps dwarf the 0.3 mm height RMS), 13/13 trajectory targets correct, ground-position RMS 14.1 mm, entry latency 0.2 s (the 3-frame confirmation), per-person occupancy minutes recovered to 0.1 min. Caveats: identity-by-height is valid only for small cohorts with distinct heights (it is a household biometric, not a general one), and the idealized head/torso detections make these upper bounds — real image-tier person detection is where the learned-detector boundary of subsection 9.1 sits.

14.4 What the scenarios share

All four reuse the same small kernel — epipolar gating, triangulation, model-constrained fitting, temporal confirmation — plus one scenario layer each (multi-target association; photometric step decomposition; identity classification). The common upgrades they all point to: constant-velocity Kalman filtering (speed estimates, Table 11), the Mahalanobis-weighted fits of subsection 13.1, and the background-subtraction front end of subsection 9.4.

15 Person detection and face recognition with open-source components

The learned-detector boundary drawn in subsection 9.1 does not have to remain outside the system: mature open-source detectors and face recognizers can be attached *without* touching the dependency-free core, and the occupancy scenario (subsection 14.3) already defines the exact seam they plug into. This section records the component landscape, the integration architecture, and — the question that determines what to promise — how accuracy differs between a pre-registered cohort and unknown faces.

15.1 Component landscape

Three roles, each with solid open-source fillings: *person detection* — YOLO-family single-shot detectors [RF18] (the original darknet implementation is plain C; Apache-licensed successors such as YOLOX run everywhere), or classical HOG pedestrian detection in OpenCV as a weak-but-tiny fallback; *face detection + landmarks* — RetinaFace [DGV+20] or its lightweight successors (SCRFD, YuNet), which return a box plus five landmarks (eyes, nose, mouth corners); *face*

embedding — deep metric networks mapping an aligned 112×112 crop to a unit vector such that same-person pairs are close: FaceNet established the recipe [SKP15], ArcFace-class models are the current open-source standard [DGXZ19]. The practical integration seam for all three is an ONNX-exported model behind the ONNX Runtime C API (MIT-licensed, single shared library, plain-C interface) — the one boundary object between the C99 world and the learned world. License note: several popular YOLO packagings are AGPL; the Apache/MIT alternatives keep the system's licensing simple.

15.2 Piping it in

The architecture is a *sidecar oracle*: the core stays pure C99 and treats the detector exactly as the scenario demos treat their simulated detections. Per frame and per camera, the sidecar consumes the image and emits rows of `(frame, camera, class, box, landmarks, embedding)` — a text/CSV stream over a pipe or file, trivially parseable, recordable, and replayable (determinism is preserved by recording the sidecar's output once and replaying it in tests — the oracle's answers become fixtures). The geometric core then does what it already does:

1. *stereo association*: detections in the two cameras are paired by the epipolar gate on box centers/landmarks (section 6) — the same gate that suppressed clutter in subsection 14.1;
2. *triangulation*: paired boxes give the person's 3D position; paired eye landmarks give a metric face position and, with the floor plane, the height biometric of subsection 14.3;
3. *track-level identity*: embeddings are not judged per frame but accumulated per *track* (quality-weighted average of embeddings across the track's frames): the tracker turns many marginal glimpses into one confident identity, and the identity then travels with the track through frames where the face is turned away entirely.

Identity fusion is a small score-level combination: face-embedding similarity where a face is visible, the height/gait geometric cues otherwise — the heterogeneous-evidence pattern of the appendix (subsection A.9) applied to identity.

15.3 Pre-registered vs unknown faces

The accuracy question splits into three regimes with *very* different numbers.

Verification (1:1) and closed-set identification (1:N, person guaranteed enrolled). On the classic verification benchmark LFW [HRBLM07] modern open-source models exceed 99.5% — the benchmark is saturated and mostly measures that nothing is broken. The more honest unconstrained figure is IJB-C [MAD+18] verification at strict false-accept rates: ArcFace-class models reach a true-accept rate around 96–97% at FAR $= 10^{-4}$ [DGXZ19]. For a *pre-registered household cohort* — N of order 10, controlled enrollment photos, closed-set — rank-1 identification is effectively $\geq 99\%$ whenever the probe image is adequate: with guaranteed gallery membership, the decision is an argmax among a few well-separated identities, and errors are driven almost entirely by probe quality, not confusion.

Open-set identification (the person may be unknown). This is a different problem: the system must first decide *whether* the probe is anyone enrolled, which requires thresholding an absolute similarity score rather than taking an argmax. Impostor scores from unknown faces overlap the tail of genuine scores, so every operating point trades false alarms (visitor labeled as a resident) against misses (resident labeled unknown). NIST's identification evaluations [GNH19] quantify the cost: even leading algorithms give up substantial accuracy relative to closed-set — at a false-positive identification rate of 1%, miss rates range from a few percent with high-quality frontal probes to *tens of percent* with surveillance-quality probes, and degradation grows with gallery size and probe–gallery quality gap. The rule of thumb this system should adopt: **closed-set household identification is a solved problem at our scale; reliable "this is a stranger" detection is not free** — it needs a margin-calibrated threshold, benefits enormously from track-level embedding fusion, and should be reported with its false-alarm/miss operating point, never as a single accuracy number.

The resolution gate. All of the above presumes enough face pixels, and here our own subsection 10.3 bites hard.

Example 21 (face pixels at range)**.** A face is $\approx 0.20\,\text{m}$ tall with $0.064\,\text{m}$ interocular distance. At the experimental rig's $f = 800\,\text{px}$ and $Z = 4.7\,\text{m}$: face height $0.20 \cdot 800/4.7 \approx 34\,\text{px}$, interocular $\approx 11\,\text{px}$ — far below the $\gtrsim$ 80–112 px face crops the embedding networks are built for; expect severely degraded embeddings. For a $100\,\text{px}$ face at $4.7\,\text{m}$ the rig needs $f = 100 \cdot 4.7/0.20 = 2350\,\text{px}$ — a 4K-class sensor, or a normal sensor with a narrow-field lens.

The systems answer follows the appendix's blending argument (subsection A.9): do not demand identity everywhere. Add one *identity camera* — a narrow-field unit aimed at the natural chokepoint (the door of subsection 14.3, where every track begins and ends) — capture the face at high resolution once per entry, and let the tracker carry that identity through the wide-angle geometric volume. Identity becomes an attribute assigned at a favorable moment and *propagated by geometry*, rather than a per-frame recognition burden. A final scope note befitting the controlled environment: the enrollment gallery is the household's own, processing is local, and the open-set threshold should be set to prefer "unknown" over misattribution.

15.4 Plugin matrix: OSS solutions, features, and license isolation

Table 15 surveys the practical candidates. Feature columns: person detection, face detection, landmarks, embeddings, ONNX exportability (i.e. runs behind the single ONNX Runtime seam), and CPU-only viability. The last column states the *required* degree of separation to avoid license contamination of the MIT-clean core:

- **link** — permissive license (MIT/BSD/Apache/BSL): may be linked into the same process; no obligations beyond attribution.
- **process** — AGPL: never link or import; run as a separate OS process communicating *data only* (frames out, text rows back). The combined system must not be distributed as one work, and if the sidecar is ever offered as a network service, AGPL source obligations attach to the sidecar alone.
- **weights apart** — code is permissive but the pretrained *weights* carry a research/non-commercial license: treat weights as user-fetched artifacts at install time, never redistributed with the system.

Architecturally we put *every* detector behind the sidecar protocol of subsection 15.2 regardless of license — hygiene first — so the license question collapses to "which side of the pipe, and who ships the bytes."

Solution	License (code/weights)	Person	Face	Lmk	Embed	ONNX	CPU	Isolation
darknet YOLOv3/v4	MIT-like / same	✓	–	–	–	part	slow	link
YOLOX	Apache / Apache	✓	–	–	–	✓	ok	link
Ultralytics v8/v11	AGPL / AGPL[a]	✓	–	pose	–	✓[a]	ok	**process**
OpenCV (HOG/YuNet/SFace)	Apache / Apache	weak	✓	5pt	✓	n/a	✓	link
MediaPipe	Apache / Apache	pose	✓	✓	–	–	✓	link
dlib / face_recognition	BSL / public	HOG	✓	68pt	✓	–	slow	link
InsightFace (SCRFD+ArcFace)	MIT / research[b]	–	✓	5pt	✓	✓	✓	weights apart
facenet-pytorch (MTCNN+FaceNet)	MIT / MIT	–	✓	5pt	✓	✓	ok	process[c]
ONNX Runtime (the seam)	MIT	— inference runtime for all ONNX rows —						link

Table 15: OSS plugin candidates. [a]AGPL applies to exported/converted weights too — conversion does not launder the license; mandatory process isolation, data-only IPC, no combined distribution. [b]InsightFace *code* is MIT but the model-zoo weights are research/non-commercial: fine in this controlled personal deployment, user-fetched at install, never redistributed. [c]License permits linking; the Python runtime makes a separate process the practical choice anyway.

Table 16 adds the published quality metrics needed to actually decide. Conventions: person detection is COCO $\text{mAP}_{50\text{–}95}$ (small→large model variants as a range); face detection is WIDER FACE validation AP for the easy/medium/hard splits (hard is the discriminative one — small, blurred, occluded faces, i.e. our at-range regime); embeddings are LFW verification [HRBLM07] (saturated; a sanity floor) and IJB-C TAR at FAR = 10^{-4} [MAD+18] (the honest unconstrained number). "n/p" = no comparable published figure.

Solution	COCO mAP	WIDER e/m/h	LFW	IJB-C TAR@10^{-4}
darknet YOLOv3 / v4	33.0 / 43.5	–	–	–
YOLOX (s→x) [GLW+21]	40.5–51.1	–	–	–
Ultralytics v8/v11 (s→x)	44.9–54.7	–	–	–
OpenCV HOG	legacy[a]	–	–	–
OpenCV YuNet / SFace	–	≈ 89/87/77	99.60	n/p
MediaPipe (BlazeFace/Pose)	n/p[b]	n/p[b]	–	–
dlib / face_recognition	–	n/p	99.38	n/p
InsightFace SCRFD-10GF [GDLZ21]	–	≈ 95/94/83	–	–
InsightFace RetinaFace-R50 [DGV+20]	–	≈ 97/96/92	–	–
InsightFace ArcFace-R100 [DGXZ19]	–	–	99.83	≈ 96–97
facenet-pytorch (MTCNN / FaceNet)	–	≈ 85/82/61	99.63	n/p

Table 16: Published quality metrics, as reported by the respective authors and model zoos (values rounded; model-zoo figures marked ≈). [a]Pre-CNN pedestrian detection; an order of magnitude behind modern detectors on all benchmarks, listed only as a zero-dependency fallback. [b]Optimized for mobile near-range operation; not benchmarked in comparable terms.

Reading the two tables together, the decision logic is: the license-clean default stack — YOLOX (person), SCRFD or YuNet (face), SFace or user-fetched ArcFace (embedding), all as ONNX graphs behind ONNX Runtime — concedes very little. On person detection YOLOX-m (46.9) sits within a few mAP of the AGPL-fenced Ultralytics models, nowhere near a factor that survives our track-level temporal fusion. On face detection SCRFD trails RetinaFace-R50 mainly on the hard split (83 vs 92) — relevant, since at-range faces *are* the hard split, but the identity-camera design (Example 21) moves recognition to the easy split at the chokepoint.

On embeddings the gap that matters is SFace (99.60 LFW, no published IJB-C) vs ArcFace (99.83 LFW, ≈ 96–97 IJB-C): for *closed-set* household identification both are past the point of mattering; for *open-set* stranger detection (subsection 15.3) the strongest available embedding is the one to fetch, which argues for ArcFace weights as the user-supplied artifact. MTCNN's 61 on WIDER-hard is the reason it is not the default despite MIT-clean weights. One standing caveat, in the spirit of subsection 13.2: all these numbers are measured on academic benchmarks at favorable resolutions — they *rank* the candidates but do not predict absolute in-system accuracy in our at-range regime, so the integrated sidecar gets its own acceptance test (recorded-oracle replays scored against identity-camera ground truth) before any number from this table is quoted as a system property. The Ultralytics row exists because its tooling is convenient for experiments — inside its own fence.

15.5 Deployment architecture: hardware boundaries and communication

Live rig v1 (built). The streaming skeleton of this architecture now exists and has been soak-tested in loopback: `tools/stream_cam.swift` (macOS capture) and `tools/stream_cam_v4l2.c` (Linux capture, same wire format) send grayscale frames over TCP to `tools/livehub.c`, which drains the network on one thread while a single decoder thread (the reader is non-reentrant) always works on each camera's freshest frame, running the full estimator chain *live*: blind decode (both tiers) → robust Zhang → LM refinement → counter-matched extrinsics, with session records and decoded frames persisted for the offline full-precision pass. `tools/replaycam.c` replays recorded frames over the identical protocol (the loopback tester), and `tools/nettest.c` is a deterministic protocol-torture client. The first soak runs paid for themselves immediately: TCP backpressure from multi-second decodes throttled live senders (fixed by the thread split), block-buffered logs vanished on kill (fixed), glibc's feature-test macros broke the Linux build invisibly to macOS (fixed, found by the `deploy/` container audit), and the fuzzer's review yielded a quadratic-resync CPU DoS, slot-exhaustion lockout, and unchecked-allocation crashes, all fixed and re-fuzzed. Live loopback calibrations match the offline pass within 0.6%.

The communication patterns are deliberately boring: hardware trigger pulses for simultaneity (subsection A.2); USB3 raw frames into the metric host; a one-directional frame stream to the sidecar (shared memory locally, TCP if the sidecar moves to its own machine); text rows back. Nothing crosses the license boundary except data, every byte crossing it is recordable — which is exactly what makes the sidecar replayable in tests (subsection 15.2) — and the C99 core never links against anything but `libm`.

Figure 9 gives the complementary *machine-centric* view: what software and persistent data each physical box holds, and every inter-machine data flow. Two locality rules are worth making explicit. First, *scene and identity data rest only on the metric host*: the inference box holds model weights but stores no frames, no tracks, and no gallery — faces cross to it only transiently for embedding, and the similarity decision is made back on the trusted host. Second, *sensors and fixtures are stateless*: cameras, display, and plate hold nothing persistent, so replacing any of them is a hardware swap plus recalibration, never a data migration.

16 Roadmap

Dependency-ordered. Each entry gives a blurb (what it is) and a pitch (why it fits this system now); statuses reflect v0.4.0.

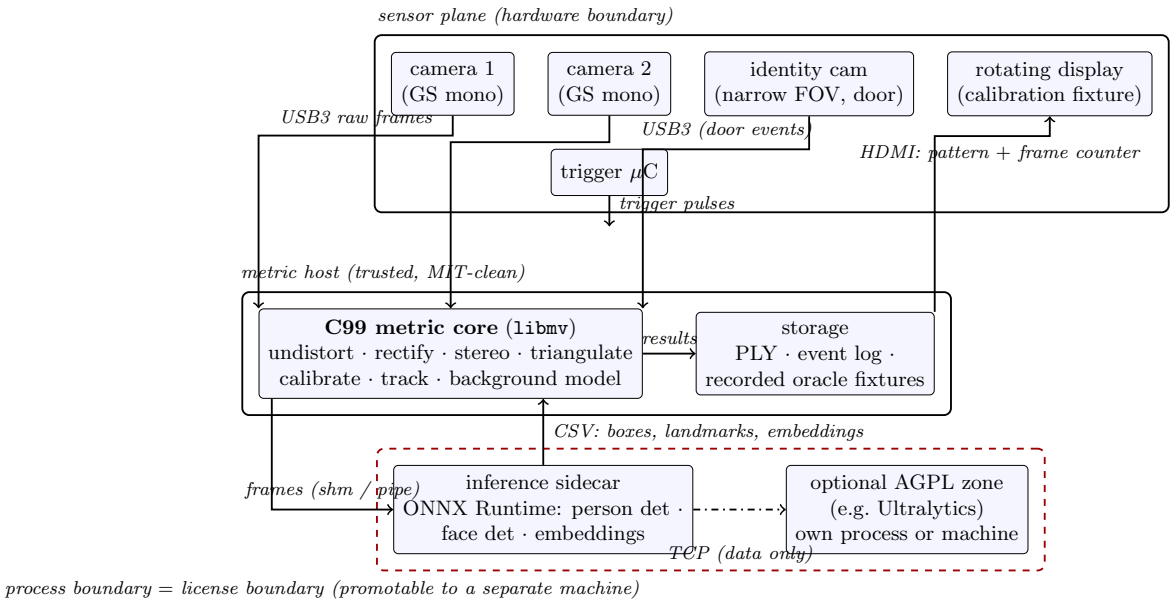


Figure 8: Deployment architecture. Solid boxes are hardware boundaries; the dashed red box is the license/process boundary — all learned components live behind it, exchanging only data (frames one way, text rows back), so it can be promoted from "separate process" to "separate machine" (a small GPU/NPU box) without any protocol change. The trigger microcontroller synchronizes both cameras, the identity camera, and — during calibration — the rotating display's frame counter.

1. **[done] Saddle-point corner detection** — implemented in `src/reader.c`; setup and run instructions are in the comment atop `mv/reader.h` (`make demo_patternsim`, `MV_READER_DEBUG=1` for tracing). Sub-pixel template detection, validated blind through the rendered pipeline at 0.25–0.35 px (Table 7). *Unlocks*: the image-tier calibration chain that every entry below assumes.

2. **[partially done] Calibration subsystem completion** — built so far: pattern generator (both tiers), synthetic renderer, blind readers (`mv/pattern.h`, `mv/render.h`, `mv/reader.h`), the display app itself (`tools/pattern.html`, modes M0–M4, vsync counter, Retina-aware 1:1 pixels), the robust video estimator (subsection 5.4), the single-frame localizer `mv_calib_plane_pose`, and the rig-extrinsics tool `tools/rigcalib.c` (counter-matched simultaneous views → chordal-mean relative poses). All components field-proven, rigcalib included: the round-five capture produced the first real camera-to-camera extrinsics (0.330 m baseline at 2.3°/6 mm pair consistency). The record-emitting wrapper, version-block check, phase-patch sampling, and the robust Theil–Sen clock fit are now built (`mv/session.h`, grammar per Table 4, round-trip tested). Remaining: only the joint orbit fit, which waits on the rotating fixture hardware. *Pitch*: this turns the fixture design into a push-button session — one revolution in, geometry and clocks out — and every missing piece already has its spec (subsection 5.9) and its acceptance statistic waiting; nothing needs inventing, only building. *Unlocks*: real-rig deployment of every scenario in section 14 (all assume a calibrated, clock-modeled rig), and the M0 radiometric path that the lighting/diurnal scenario (subsection 14.2) needs on real hardware.

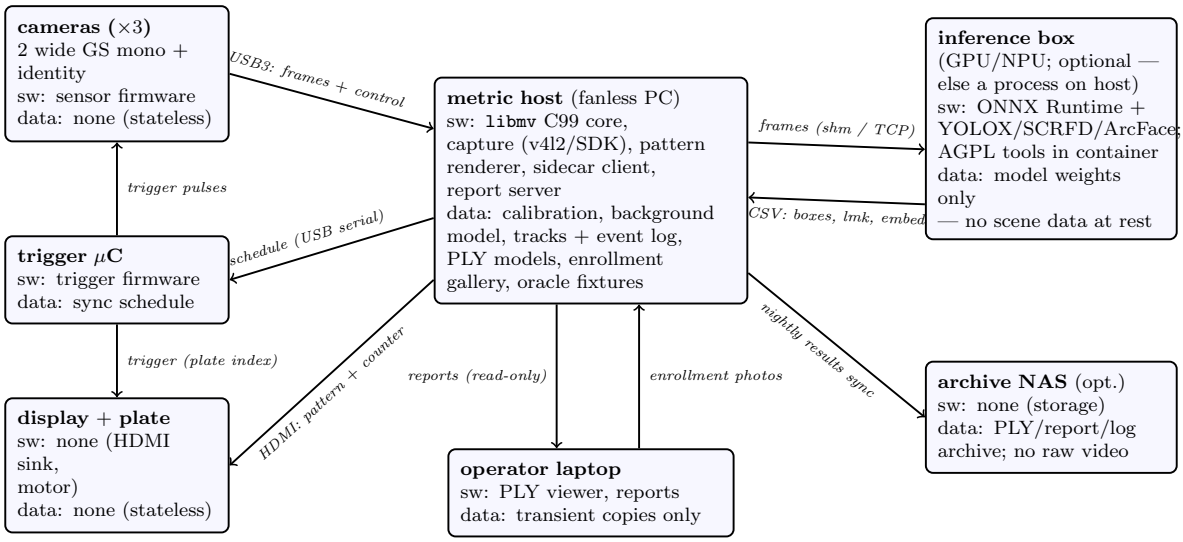


Figure 9: Machine-centric deployment view: per-machine software and persistent data, and all inter-machine flows. Scene and identity data rest only on the metric host; the inference box is compute-only; sensors and fixtures are stateless.

3. **[done] Range-tiered pattern (spec v2)** — the coarse layer for room-scale reading, implemented as display mode M4 (subsection 5.10): 270 px cells, 90 px marks, isolated-square 8-bit counter — ∼3.4× the range of (19) at 10 corners per frame instead of 162. Generator `mv_pattern2_render` (`mv/pattern.h`), reader `mv_read_coarse` (`mv/reader.h`), consumed by `tools/calibreal.c` and `tools/rigcalib.c` as an automatic fallback; validated by `test_pattern_coarse` at the exact field-failure geometry. *Pitch*: round four measured the need directly — three fixed cameras watching a room could find the saddle lattice but not one identity dot. *Unlocks*: rig calibration at room scale with commodity webcams — the actual deployment scenario of section 2. The temporal multiplexing refinement is built: display mode M6 alternates the tiers in 64-refresh blocks (`mv_pattern_mux_tier`, `mv_pattern_mux_render`; `tests/test_mux.c` round-trips both tiers from one stream), so near and far cameras share a single capture on one counter timeline (fine reads give k in full, coarse reads k mod 256, reconciled wrap-aware as `rigcalib` already does).

4. **[done at sim tier] Projected structured light** — subsection 11.2: display-app mode M5, cross-camera code matching (`mv_graycode_match`), robust 8-point + essential pose (`mv_essential_pose`), the `slreal` tool, and the `demo_slight` integration sim that fabricates a full two-camera capture and recovers the ground-truth pose to the fourth digit. *Pitch*: rooms are mostly blank wall, and blank wall is where our dense stereo honestly reports nothing; a $60 projector paints structure onto exactly those surfaces, and doubles as an extrinsics bridge for camera pairs that cannot both see the display. *Unlocks*: dense room geometry on textureless surfaces; awaiting only the projector hardware — the runbook in subsection 11.2 is written for solo operation.

5. **[done] Levenberg–Marquardt refinement** — implemented as `mv_calib_refine` (`mv/refine.h`): joint LM over shared intrinsics/distortion and per-view Rodrigues poses with a Schur complement over the view blocks; the pre-registered T5 acceptance passed (ensemble ratio

1.1087 → 0.9990, subsection 13.2); exact at $\sigma = 0$; never worse than the alternation initializer on any ensemble member. Bundle adjustment over registered views is also built (`mv/bundle.h`: multi-camera + points joint LM, Schur complement over per-point blocks, gauge anchored by flagged fixed cameras): residual ratio 1.012 against the predicted floor, and it tightens photo registration from 77 mm to 6 mm while recovering the true principal point that resection had to pin. A finding worth keeping: with the v1 single-scale features, the rig's anchor bank came out 100% coplanar (all on the fronto-parallel wall; grazing planes contributed zero anchors), and free-K bundle adjustment over a coplanar set walks the focal–depth degeneracy to a spurious lower-residual solution — the same degeneracy that justified pinning the principal point in subsection 11.1, now measured a second way; depth relief in the anchor set is a precondition, checked in the test. (Earlier framing: bundle adjustment [TMHF00]). Joint nonlinear minimization of reprojection error over intrinsics, poses, and distortion. *Pitch*: the single most attractive item on the list — the fired T5 flag of subsection 13.2 is a *pre-registered acceptance test* (the residual ratio must fall from 1.17 to 1.00 ± 0.03, else the implementation is wrong); it tightens every calibration number in section 13, and the same LM core serves the orbit fit, the clock fit, and eventually bundle adjustment. *Unlocks*: no new scenario — it upgrades all of them; tightened $\mathbf{K}$ and clocks propagate into every tracking and model figure, and the drift monitor gains sensitivity.

6. **[done] Feature detection and robust matching** — `mv/feat.h`: Shi–Tomasi corners with sub-pixel refinement, orientation-normalized 8×8 patch descriptors, Lowe-ratio + mutual matching, verified through the robust 8-point: 219 verified matches at 0.117 px median epipolar residual against the analytic $\mathbf{F}$ in sim, 176 under a 20° roll; single-scale by design (v1). General interest points with descriptors, matched under the RANSAC gate of section 6. *Pitch*: unlocks everything "live" — the $\mathbf{F}$-drift and Δt monitors get real correspondences to chew on, and movers become trackable from images rather than simulated detections. *Unlocks*: live insect counting and robot tracking (subsection 14.1, Table 11) on real footage, and the run-time Δt estimator of subsection A.3.

7. **[done at sim tier] Rectification warping and the dense path** — `mv_warp_homography` and the multi-plane scene renderer (`mv/render.h`); run notes atop `demo/roomsim.c` (`make demo_room`). Validated in Table 9/Figure 5: the pipeline of section 2 runs on pixels end to end — and now on *real* frames too: `tools/scenecloud.c` ran the same path on handheld phone video to produce the 82k-point cloud of Figure 6. Remaining at real tier: multi-camera (fixed-rig) depth rather than accidental baselines. *Unlocks*: dense change detection and moving-object silhouettes; with items 6 and 9, the room model — demonstrated on rendered imagery and, as a point cloud, on real imagery.

8. **[done at sim tier] Temporal statistics and the background model** — median-depth background, TSDF fusion, and change detection implemented in `demo/roomsim.c` (Table 9: 97.9% recall, 1.39% false alarms on a moved box). Remaining: promotion from demo to library module, incremental background updates, and real frames. *Unlocks*: the furniture-level room mesh and scene-rearrangement events — demonstrated — and the subtraction front end for the section 14 trackers.

9. **[done at sim tier] Photo-collection registration** — subsection 11.1: `mv/photo.h` resects casual unknown-camera photos into the rig frame from a triangulated anchor bank

(RANSAC + trim + full-LM polish, principal point pinned per casual-photo convention); sim: focal within 2.6%, orientation 0.25°, position 0.11 m. Formerly: (subsection 11.1). Resection of auxiliary photos against triangulated structure; N-view DLT already accepts the extra views. *Pitch*: coverage where the rig cannot see (the topology argument of subsection 10.2) — and its dynamic limit is the tracked mobile camera with measured poses, the self-tracking extension of section 1. *Unlocks*: high-fidelity topology capture (the coverage argument of subsection 10.2) and the drone / self-tracking scenario itself.

10. **[done] Optimal and weighted estimators** — `mv/optimal.h`: Hartley–Sturm optimal triangulation (polynomial minimization, exact epipolar consistency to 10^{-13} px; the honest sim finding is that for our well-conditioned rig it matches plain DLT to 0.01% — consistent with Hartley–Sturm's own assessment) and the Mahalanobis-weighted known-radius circle fit that retires the documented $\sigma = 0.6$ px nonlinearity (1.4× lower center error; an intermediate IRLS design was measured *worse* than unweighted and rejected — exact ML on the anisotropic distance was required). Hartley–Sturm triangulation [HS97], Mahalanobis-weighted model fits, weighted DLT (subsection 10.4). *Pitch*: retires the *documented* estimator floors — the circle-fit anisotropy of Table 13 and the equal-weight DLT — each with its regression already measured, so improvement is provable, not aspirational. *Unlocks*: mover tracking in the regimes the current fits leave (Table 13) — fast insects at range, robots in dim light — and honest fusion of unequal cameras (subsection 10.4).

11. **[done at sim tier] TSDF completion** — implemented in `src/tsdf.c`; setup and run instructions are in the comment atop `mv/tsdf.h` (`make demo_tsdf`; mesh lands in `out_tsdf.ply`); validated in Table 8. Remaining: the dense-depth producer (item 5), the voxel-major fast path, and then RANSAC plane/primitive extraction on the background model. *Pitch*: every remaining piece rides on item 5 or 6; primitives (subsection 9.1) then give the furniture-level scene description almost for free. *Unlocks*: continuous room-model maintenance and geometric change detection as a standing application beside the section 14 trackers.

12. **The identity sidecar** (section 15). Person and face detection behind the ONNX Runtime seam, replayed oracles as fixtures. *Pitch*: the contract is fully specified down to the license table and quality metrics (Table 15–Table 16); it upgrades occupancy from height-cohort identification to named persons at the door chokepoint, with its acceptance test (subsection 15.3) already defined. *Unlocks*: named-person occupancy timelines and stranger detection.

If the hardware materializes, the optional range-finder integration of subsection 11.3 slots in without redesign. A suggested near-term order: item 3 first (its acceptance test is pre-registered and it uplifts everything already built), then 2 (completes the calibration story), then 5 — after which 6 and 9 are largely composition and the system produces its first real-image room model. Each step lands as a new module behind the same conventions (Table 1) with its own noiseless-exact tests and a synthetic experiment extending section 13.

Table 17 inverts the list: for each application scenario in the document, which roadmap items unlock it, what can be developed and validated in simulation alone, and what genuinely requires hardware.

Scenario	Items	Sim-only development	Requires hardware
robot tracking (Table 11)	2, 4	*done* (tracker + fits)	rig + live detections
insect counting (subsection 14.1)	2, 4, 8	*done* (tracker, gating, stats)	rig; detection is at image tier
occupancy, height-ID (subsection 14.3)	2, 4, 5	*done* (tracking, ID, paths)	rig + live detections
lighting / diurnal (subsection 14.2)	2	*done* (analyzer, 120-day synthetic)	calibrated camera + M0; real lamps
calibration session (subsection 5.9)	2, 3	reader *done*; orbit/clock fits simmable	display, plate, cameras
room model + change detection (subsection 9.3)	5, 6, 9	*done* (Table 9)	rig for real-image depth
topology capture (subsection 10.2)	7	fully simmable (rendered scenes)	photos of real objects
named-person identity (section 15)	2, 10	fusion logic, synthetic embeddings	identity camera + sidecar; real faces
drone / self-tracking (section 1)	4, 7, 8	fully simmable (mover carrying a sensor)	drone + rig

Table 17: Scenario unlock map. "Items" are the roadmap entries above; *done* marks sim-tier validation already reported in section 13–section 14. Every scenario's *estimation* layer is sim-developable; hardware enters for real detections, real radiometry, and real faces.

A Camera hardware and the quality of data

The capture hardware determines which error terms enter the pipeline before any algorithm runs. The organizing observation: cheap capture hardware mostly adds *biases* — rolling shear, synchronization parallax, compression structure, unmodelable distortion, focus creep — rather than noise. Noise we model and budget (Example 14); biases survive averaging, masquerade as geometry, and poison calibration silently. The specifications below are therefore ranked by which biases they admit.

A.1 Shutter: global vs rolling

A global-shutter sensor exposes all pixels over one common interval — each frame is a true snapshot, described by a single $\mathbf{P}$. A rolling-shutter sensor exposes and reads row by row: the bottom row of a frame is captured 10–30 ms after the top row. For a static scene the difference vanishes; for anything moving, each row samples a different instant and no single projection matrix describes the frame — the camera model of section 4 itself breaks.

Example 22 (rolling-shutter shear on the rotating target)**.** The rotating display of subsection 5.7 moves at ≈ 27 px/s in the image. A rolling readout of 20 ms samples the top and bottom of the target 20 ms apart: a shear of $27 \cdot 0.02 \approx 0.5$ px across the corner grid — larger than the 0.3 px noise budget, and *systematic*: it does not average away over frames but feeds a consistently wrong homography into calibration.

A.2 Synchronization: hardware trigger vs free-running

A trigger line starts exposure on all cameras from one electrical pulse (coincidence to microseconds). Free-running cameras can only be matched by software timestamps — a few *milliseconds* of jitter and drift. Triangulation assumes both cameras saw the point at the same instant; a time offset Δt on a point moving at speed v creates a phantom disparity $f\, v\, \Delta t / Z$ that is mathematically indistinguishable from depth.

Example 23 (the price of software synchronization)**.** An object crossing the experimental rig at 1 m/s with 5 ms software sync: phantom disparity $= 800 \cdot (1 \cdot 0.005)/4.7 \approx 0.85$ px, which the depth law (14) converts to ≈ 47 mm of depth error — exceeding the rig's entire 23 mm error budget at that range. For dynamic scenes, synchronization error dominates every other term; a

trigger wire outperforms any camera upgrade. The trigger also permits phase-locking exposure against mains flicker and locking capture to the display's frame counter (subsection 5.7).

A.3 Software synchronization: measuring and correcting the clocks

This subsection is the practice counterpart of the temporal camera model (5) of subsection 4.1: how the clock parameters (ϕ_i, T_i) are actually measured and their effect corrected. When no trigger line exists (the consumer tier), frames from independent, possibly heterogeneous cameras are never simultaneous: free-running 30 fps streams differ by up to half a frame period ($|\Delta t|$ averaging ∼8 ms), and consumer crystals drift 50–100 ppm, so the offset slides several milliseconds per minute — it must be *tracked*, not calibrated once. Since Example 23 showed the cost lands entirely on movers (static structure is indifferent to Δt), the right treatment is not to chase alignment but to make time part of the measurement model. Three layers:

Bound. Pair frames nearest-neighbor in time rather than by index: free, and caps $|\Delta t|$ at half a period under any drift.

Measure. Model each camera's clock as offset + skew against a common reference and estimate both continuously, two independent ways. (i) *The dynamic display pattern* (subsection 5.7): the Gray-coded counter gives frame-level display timestamps (record `CTR` of Table 4); the phase patches refine them below the refresh period (`PHS`); a linear fit of (camera frame index, decoded display time) pairs per camera yields its offset and rate — see the precision budget in Example 24. (ii) *Movers themselves*: a tracked point moving across the epipolar field sits off its epipolar line by (image velocity $\perp$ line) $\times \Delta t$; with $\mathbf{F}$ known exactly, Δt is a one-parameter fit to residuals already computed — the $\mathbf{F}$-drift monitor of section 6 with one extra parameter, usable at run time when no display is present. (A μC-driven blinking LED in a frame corner is the minimal permanent clock beacon if continuous display-free measurement is wanted.) A deliberate small rate offset on one camera (∼0.1 Hz) makes the phases *beat*, sweeping Δt through zero every ∼10 s: the "golden pairs" at each null (aligned to ∼50 μs) are free validation samples for the estimator.

Correct. Give every measurement its own timestamp and interpolate *tracked detections* (not images) onto the common clock before triangulating; the residual is second-order (Example 25). Dense stereo needs no correction at all — the dense content is the static background; movers travel the tracked path, which is exactly the path that interpolates. The principled limit for fast movers is trajectory triangulation: fit a short motion model to K timestamped rays rather than intersecting two rays at a fictitious common instant — and the same per-measurement-timestamp machinery is what rolling-shutter correction (subsection A.1) consumes, so building it once serves both. Self-verification in the subsection 13.2 style: the slope of epipolar residual vs image speed must equal the fitted Δt, and golden pairs must show none.

Example 24 (clock-fit precision from the display)**.** Display timestamps are quantized to the refresh period (16.7 ms at 60 Hz), i.e. per-sample noise $\sigma_q = 16.7/\sqrt{12} \approx 4.8$ ms. One minute at 30 fps gives $N = 1800$ samples: offset precision $\sigma_q/\sqrt{N} \approx 0.11$ ms, and rate precision $\sigma_q\sqrt{12/N}/T \approx 7$ ppm over the $T = 60$ s span — comfortably resolving 50–100 ppm crystals in one minute, before the phase patches sharpen σ_q further.

Example 25 (interpolation makes sync second-order)**.** A walker at 1 m/s with a full half-frame mismatch ($\Delta t = 16.7$ ms) uncorrected: phantom depth error ≈ 150 mm by Example 23. Linearly interpolating the tracked detection between frames leaves only the curvature term $a\,\Delta t^2/8$: at $a = 2\,\mathrm{m/s^2}$, $2 \cdot 0.0167^2/8 \approx 70\,\mu$m — a factor of $\sim$2000. If Δt itself is known to ± 1 ms (Example 24), the residual first-order term is $\sim$1 mm at 1 m/s: software synchronization turns the dominant dynamic-scene error into a minor one.

The verdict, stated as a rule: on the consumer tier, synchronization mismatch is quantified and corrected, never assumed away — one more instance of this appendix's theme that a *modeled* defect becomes a term of the estimator rather than a bias. The trigger tier (subsection A.2) remains the clean answer where the hardware allows it.

A.4 Radiometric and noise measurement procedures

The cameras are the rig's own best measuring instruments; these four procedures, run once per rig (modes M0/flat scenes, Table 3), produce the numbers everything else consumes.

1. *Photon-transfer curve* → the noise scale σ. Capture repeated frames of a static scene at several exposure levels; per-pixel temporal variance vs mean is linear with slope 1/gain (e^-/DN) and intercept the read-noise floor. The resulting intensity noise, propagated through the matcher, is the rig's *measured* σ — the number that re-scales every acceptance band of subsection 13.2 (which this document evaluates at the synthetic 0.3 px).

2. *Flicker audit.* High-rate (or exposure-phase-swept) capture of a blank wall; the FFT of mean intensity at $2f_{\text{mains}}$ (100/120 Hz) gives the modulation depth. Rule: keep it below $\sim$1–2%, or set $t_e = n/(2f_{\text{mains}})$, which nulls it exactly.

3. *Inter-camera radiometric transfer.* A gray card (or the display's M0 ladder) viewed by both cameras; the joint histogram gives the gain/offset map between them. Flat and linear clears plain SAD matching; visible curvature is the signal to switch the matcher to a rank/census cost.

4. *Flat-field / vignetting.* The display showing uniform gray (M0), imaged defocused; the per-pixel gain map is stored and divides all subsequent frames; it also supplies the local white references the reader's decoding rules assume (subsection 5.9).

A cheap lux meter anchors the ladder in absolute units so results transfer between sites; the display's off-axis brightness falloff — measured for free as the plate sweeps (subsection 5.7) — completes the radiometric picture of the fixture itself.

A.5 Interface: UVC vs machine-vision stacks

UVC (USB Video Class) is the "any webcam just works" protocol (`v4l2` on Linux, no vendor SDK). Its convenience reflects its purpose — streaming video to humans: at higher resolutions frames usually arrive *MJPEG-compressed*, and JPEG's 8×8 block artifacts are structured noise at exactly the spatial frequencies where sub-pixel corners and block matching live — a *correlated* contribution to σ_d that does not average like photon noise. UVC also offers only coarse loosely-specified exposure/gain controls, imprecise timestamps, and no trigger support in the class. Machine-vision interfaces (USB3 Vision/GenICam via vendor SDKs) deliver raw pixels, physical-unit controls, per-frame hardware timestamps and trigger I/O. Rule of thumb: UVC is acceptable exactly where rolling shutter is acceptable — static bring-up.

A.6 Optics: M12 vs C/CS mounts

M12 ("S-mount") lenses are the small threaded board lenses of webcam-class hardware: cheap, fixed aperture, focused by rotating the barrel, no standardized flange distance. C-mount and CS-mount are the machine-vision standards: metal barrels, multi-element glass, iris and focus rings with lock screws, standardized flange distances (C: 17.53 mm; CS: 12.53 mm; a C lens fits a CS body via a 5 mm spacer, not conversely). Three separate data-quality channels:

1. *Sharpness (MTF) is the true floor under σ_d*: sub-pixel localization precision scales with the optical blur radius; a soft lens can double σ_d, and via (14) double σ_Z — equivalent to discarding half the sensor resolution (cf. the weakest-link law of subsection 10.4: the lens is a full member of that chain).

2. *Distortion modelability*: the Brown model (4) absorbs the smooth distortion of decent lenses; cheap wide M12s exhibit higher-order or non-radial terms that leave *systematic* calibration residuals.

3. *Stability of* **K**: thread-focused plastic barrels creep with vibration and temperature — silent recalibration events, precisely what the **F**-drift monitor of section 6 exists to catch. Locked C/CS rings hold **K** for months. Autofocus is this failure mode as a feature: *a calibrated camera must never refocus.*

An iris adds one more useful knob: most lenses peak in sharpness around f/4–f/8; beyond that diffraction itself blurs (spot $\approx 1.22\,\lambda N$: at f/8 about 5.4 μm, already exceeding a 3 μm pixel).

A.7 Monochrome vs color

A Bayer color sensor samples each color at $\frac{1}{4}$–$\frac{1}{2}$ of the pixel grid and interpolates the rest; demosaicing smears exactly the high-contrast edges that corners and matching depend on, and costs about one stop of sensitivity. This pipeline processes grayscale throughout, so monochrome sensors deliver strictly better data; color remains at most a redundancy layer in the calibration pattern (subsection 5.7).

A.8 Summary and candidate hardware

Characteristic	Data-quality consequence	When it matters
rolling shutter	∼0.5 px systematic shear (Example 22)	any motion
no trigger	phantom disparity, ∼47 mm (Example 23)	dynamic scenes
UVC / MJPEG	structured artifacts in σ_d; no trigger	beyond bring-up
M12 optics	MTF floor, distortion residuals, **K** creep	always, silently
C/CS optics	sharp, modelable, mechanically stable **K**	the metric rig
color (Bayer)	demosaic edge artifacts, −1 stop sensitivity	mono preferred

Table 18: Capture-hardware characteristics and their consequences.

Three procurement tiers follow (indicative 2026 prices):

1. *Bring-up* (∼$150/pair): two Logitech C920/C930-class UVC webcams. Rolling shutter, no trigger, but focus and exposure lockable via `v4l2-ctl`; teaches the pipeline every real-world lesson cheaply. Algorithms yes, metric claims no.

2. *Budget metric rig* (∼$150–250/pair): OV9281-class modules (e.g. Arducam) — 1280×800 *monochrome global shutter*, external-trigger variants, interchangeable M12 optics (choose the best lens, lock it with thread-lock). With a 2.8–4 mm lens, $f \approx 950$–1300 px — the regime of all worked examples here. Same idea on CSI: the Raspberry Pi Global Shutter Camera (IMX296, trigger pin, ∼$50).

3. *Solid metric rig* (∼$500–900/pair): entry machine-vision USB3 (FLIR Blackfly S, Basler ace/dart, or Daheng/HIK equivalents): 1.6–2.3 MP mono global shutter, real trigger I/O, Linux SDKs, C/CS lenses. The tier where the resolution mathematics of subsection 10.3 becomes purchasable.

A packaged shortcut: the Luxonis OAK-D class devices (∼$250) contain two factory-synchronized global-shutter mono cameras in one USB3 housing — the whole hardware problem in one purchase, at the cost of a fixed ∼7.5 cm baseline, short for a 4–6 m working volume ($\sigma_Z \propto 1/B$, (14)).

Table 19 lists concrete representatives, sorted by price; prices are indicative mid-2026 street prices per camera and drift, but the *characteristics* are fixed properties of each product.

Model	≈$	Sensor / resolution	Mono	Global shutter	HW trigger	Raw frames	Lens
Raspberry Pi GS Camera[a]	50	IMX296, 1456 × 1088	–[a]	✓	✓	✓	CS
Logitech C920s[b]	70	1920 × 1080	–	–	–	–[b]	fixed
Arducam OV9281 USB	90	OV9281, 1280 × 800	✓	✓	✓	✓	M12
HIKrobot MV-CS016-10UM	230	IMX273, 1440 × 1080	✓	✓	✓	✓	C
Luxonis OAK-D[c]	250	2×OV9282, 1280 × 800	✓	✓	–[c]	✓	fixed
Basler dart daA1280-54um	280	1280 × 960	✓	✓	✓	✓	S/CS
Intel RealSense D435[d]	330	2×OV9282 IR	✓	✓	–[d]	✓	fixed
FLIR Blackfly S 16S2M-CS	465	IMX273, 1440 × 1080	✓	✓	✓	✓	CS

Table 19: Representative commercial cameras, sorted by indicative 2026 price per unit. [a]CSI interface (needs a Raspberry Pi, not USB); color filter — monochrome variants from third parties. [b]UVC; uncompressed only at reduced rates, MJPEG otherwise; exposure and focus lockable via `v4l2-ctl`. [c]No external trigger input, but the internal stereo pair is factory-synchronized — the trigger problem is solved *inside* the box; baseline fixed at 7.5 cm. [d]Depth device; the raw synchronized IR pair is accessible and usable as a stereo source; projector should be disabled for our matcher.

A.9 Blending heterogeneous cameras

Knowing each camera's characteristics is itself an asset: a *modeled* defect is a correctable or at least gateable one. A rolling-shutter camera whose row-readout time is known can have its rows time-stamped individually and its observations gated out (or motion- compensated) whenever the scene moves; a camera with a measured noise curve enters the triangulation with an honest weight (subsection 10.4); a lens with well-fit distortion contributes unbiased rays while one with known residuals can be trusted only in its image center. The measurement model becomes

per-camera, and every characteristic in Table 18 turns from a hidden bias into a term of that model.

A deliberate *blend* of camera types then does more than average — it makes artifacts detectable, because most artifacts are *camera-specific* while the scene is common. MJPEG block edges, rolling shear, demosaic fringes, and specular highlights each live in some cameras and not others (highlights move with viewpoint; compression artifacts sit on one camera's grid); true structure must be consistent across all views. With identical cameras, a shared systematic error is invisible — every unit votes for the same lie; with heterogeneous hardware the correlation is broken and disagreement *localizes* the artifact, in the same spirit as the **F**-drift and laser-audit health monitors (section 6, subsection 11.3). Complementary pairings fall out naturally: a synchronized global-shutter mono pair as the metric backbone; a high-resolution color camera (rolling shutter tolerated, gated by motion) contributing fine texture and appearance to the model; a wide-FOV unit for coverage and moving-object flagging, its geometry weighted down per subsection 10.4. Geometry from the cameras built for geometry, appearance from the cameras built for appearance, and cross-checks from their disagreement — the N-camera analysis of section 10 applies unchanged, since every camera carries its own **K**, distortion, and noise model.

B Root-mean-square (RMS) error: definition and use

Nearly every quantitative claim in this document is an RMS figure. This appendix defines it, explains why it is the right summary here, and maps every way the document uses it.

B.1 Definition

For scalar errors $e_1, \ldots, e_N$,

$$\text{RMS} \;=\; \sqrt{\tfrac{1}{N}\textstyle\sum_{i=1}^{N} e_i^2}, \tag{20}$$

the square root of the mean of the squares. For vector errors (a 2D reprojection offset, a 3D position error) the same formula is applied to the *norms*: $\text{RMS} = \sqrt{\frac{1}{N}\sum \|\mathbf{e}_i\|^2}$, which is also $\sqrt{\sum_k \text{RMS}_k^2}$ over the components — components add *in quadrature*.

Example 26 (RMS vs mean vs max)**.** Errors $\{1, 2, 2, 5\}$: mean $= 2.5$; RMS $= \sqrt{(1+4+4+25)/4}$ $= \sqrt{8.5} \approx 2.92$; max $= 5$. RMS always lies between mean and max, weighting large errors more than the mean does (squaring makes the 5 contribute 25/34 of the total). This document reports RMS as the headline and max separately when tails matter (e.g. Table 11).

B.2 Why squares

Three reasons, in increasing depth. (i) *Calculus*: squared error is smooth, so least-squares problems have closed forms — every estimator in this system (Example 4 onward) minimizes a sum of squares, and RMS is exactly the quantity those estimators optimize, expressed per sample. (ii) *Probability*: for zero-mean Gaussian noise the RMS of the errors converges to the distribution's σ; reporting RMS is reporting the fitted noise scale, directly comparable to the injected σ in every experiment. Least squares *is* maximum likelihood under Gaussian noise, closing the loop. (iii) *Composition*: independent error sources add in quadrature, $\sigma_{\text{tot}}^2 = \sum_j \sigma_j^2$ — the working rule behind the two-coordinate pixel distance, the two-camera angular noise (18), and the disparity noise $\sigma_d = \sqrt{2}\,\sigma$ used throughout.

Example 27 (quadrature)**.** Per-coordinate pixel noise $\sigma = 0.3\,\text{px}$: the Euclidean reprojection *distance* has RMS $\sqrt{0.3^2 + 0.3^2} = 0.424\,\text{px}$ — the baseline against which the residuals of subsection 13.2 are judged. Disparity, a difference of two independent 0.3 px coordinates, likewise has $\sigma_d = \sqrt{2} \cdot 0.3 \approx 0.42\,\text{px}$ (Example 14).

B.3 Bias, variance, and what RMS cannot tell you

RMS mixes systematic and random error:

$$\text{RMS}^2 = \underbrace{\bar{e}^{\,2}}_{\text{bias}^2} + \underbrace{\tfrac{1}{N}\sum (e_i - \bar{e})^2}_{\text{variance}}. \tag{21}$$

Example 28 (decomposition)**.** A constant bias of 3 plus zero-mean noise of standard deviation 4 gives RMS $= \sqrt{3^2 + 4^2} = 5$. Two very different failure modes, one number. This is precisely why the diagnostics of subsection 13.1 never rely on a single RMS: the σ-sweep separates the terms — variance scales with σ, bias does not — and the $\sigma = 0$ run isolates bias exactly.

B.4 Statistical handling

Two facts about RMS as an *estimate*, both load-bearing in subsection 13.2. First, a sample RMS over N Gaussian errors fluctuates with relative standard deviation $\approx 1/\sqrt{2N}$ (χ^2 with N degrees of freedom): with $N = 250$ that is 4.5%, which sets the $\pm 3/\sqrt{2N}$ null bands of the T2/T5 tripwires. Second, residuals of a least-squares fit are *smaller* than the noise, because the fit absorbs p degrees of freedom: $\text{RMS}_{\text{resid}} = \sigma\sqrt{1 - p/n}$ for p parameters and n residual coordinates. Triangulation ($p = 3$, $n = 4$) leaves $\sigma\sqrt{1/4} \cdot \sqrt{2} = \sigma/\sqrt{2}$ per-view distance — 0.212 px at $\sigma = 0.3$, matching the measured 0.218 (Table 5); calibration ($p = 42$, $n = 756$) should leave 0.412 px, and the measured 0.484 is the fired T5 flag of subsection 13.2. An RMS *below* the corrected prediction signals overfitting just as surely as one above signals underfitting.

B.5 Where this document uses RMS

- *Reprojection RMS* (px): residual image distance after triangulation or calibration — the fitted noise scale; compare to $\sqrt{2}\,\sigma\sqrt{1 - p/n}$ (Table 5, Table 6).
- *3D / position RMS* (m or mm): Euclidean reconstruction or tracking error against ground truth; components reported separately when anisotropy matters (depth vs lateral, Table 13).
- *Timing RMS* (s or min): event and sunrise timing in the scenario studies (section 14); same formula, time errors instead of distances.
- *Jitter RMS about an offset*: variance-only RMS after removing a known constant (the sunrise estimator's offset, subsection 14.2) — i.e. the second term of (21) alone, used deliberately when the bias is understood and harmless.
- *Predicted RMS*: the error laws (14), (18) are statements about the RMS of infinite repetitions; measured-over-predicted ratios are the consistency statistics of subsection 13.2.

C Where to read: the code–paper map

Two conventions keep the code and this document navigable as one work. *Code → paper*: every public header opens with a `Paper:` comment naming the section (by title, stable under renumbering) whose theory it implements. *Paper → code*: Table 20 maps each section to its implementation and to the test or experiment that validates it — so to pick up any thread, read the section, open the module, run the validator.

Paper	Implementation	Validated by
section 3, section 12	`mv/mat.h`, `src/mat.c`	SVD/inverse tests
section 4	`mv/cam.h`, `src/cam.c`	center/ray/projection tests
subsection 4.1 (clock model)	— (planned state on `mv_camera`)	Example 24 budgets
subsection 5.1–subsection 5.3	`mv/calib.h`, `src/calib.c`	Zhang exactness tests; `demo_calibrate`
subsection 5.5	`mv/target.h`, `src/target.c`	calibration tests
subsection 5.6	`mv/graycode.h`, `src/graycode.c`	round-trip tests
subsection 5.9 (pattern, spec v1)	`mv/pattern.h`, `src/pattern.c`	M-array selftest; `demo_patternsim`
subsection 5.9 (reader)	`mv/reader.h`, `src/reader.c`	blind-read test; Table 7
subsection 5.9 (validation tier)	`mv/render.h`, `src/render.c`	`demo_patternsim`
section 6	`mv/epipolar.h`, `src/epipolar.c`	F/8-point tests; `demo_synthetic`
section 7, section 10	`mv/triangulate.h`, `src/triangulate.c`	exactness tests; Table 12
section 8	`mv/rectify.h`, `mv/stereo.h`	row-alignment tests
section 9	`mv/cloud.h`, `mv/img.h`	PLY/PGM outputs of all demos
subsection 9.2	`mv/tsdf.h`, `src/tsdf.c`	sign/mesh tests; Table 8
subsection 9.3 (dense path)	`demo/roomsim.c`, `mv/render.h`	warp/scene tests; Table 9
section 13 diagnostics	`demo/diagnose.c`	the tripwires of subsection 13.2
section 14	`demo/track_robot,insects,people`; `lightlog`	their ground-truth scoring
subsection 13.2 T1	`tests/test_mv.c` (47 assertions)	`make check`

Table 20: Code–paper cross-reference. Demos print exactly the numbers quoted in section 13–section 14 (`make demo`); every experiment is deterministic and seeded.

References

[BM92] Paul J. Besl and Neil D. McKay. A method for registration of 3-d shapes. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 14(2):239–256, 1992. No local copy retrievable from open-access sources (2026-08-04). (cited on p. 28)

[CL96] Brian Curless and Marc Levoy. A volumetric method for building complex models from range images. In *Proceedings of SIGGRAPH 96*, pages 303–312, 1996. Local copy: refs/CurlessLevoy1996.pdf (retrieved 2026-08-04; screened, see refs/SCAN.md). (cited on pp. 20, 21, and 22)

[DGV+20] Jiankang Deng, Jia Guo, Evangelos Ververas, Irene Kotsia, and Stefanos Zafeiriou. RetinaFace: Single-shot multi-level face localisation in the wild. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 5203–5212, 2020. Local copy: refs/Deng2020.pdf (retrieved 2026-08-04; screened, see refs/SCAN.md). (cited on pp. 42 and 45)

[DGXZ19] Jiankang Deng, Jia Guo, Niannan Xue, and Stefanos Zafeiriou. ArcFace: Additive angular margin loss for deep face recognition. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 4690–4699, 2019. Local copy: refs/Deng2019.pdf (retrieved 2026-08-04; screened, see refs/SCAN.md). (cited on pp. 42, 43, and 45)

[FB81] Martin A. Fischler and Robert C. Bolles. Random sample consensus: A paradigm for model fitting with applications to image analysis and automated cartography. *Communications of the ACM*, 24(6):381–395, 1981. Local copy: refs/FischlerBolles1981.pdf (retrieved 2026-08-04; screened, see refs/SCAN.md). (cited on pp. 9, 17, 20, and 26)

[FP10] Yasutaka Furukawa and Jean Ponce. Accurate, dense, and robust multiview stereopsis. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 32(8):1362–1376, 2010. No local copy retrievable from open-access sources (2026-08-04). (cited on pp. 24 and 26)

[FTV00] Andrea Fusiello, Emanuele Trucco, and Alessandro Verri. A compact algorithm for rectification of stereo pairs. *Machine Vision and Applications*, 12(1):16–22, 2000. No local copy retrievable from open-access sources (2026-08-04). (cited on p. 18)

[GDLZ21] Jia Guo, Jiankang Deng, Alexandros Lattas, and Stefanos Zafeiriou. Sample and computation redistribution for efficient face detection. *arXiv preprint arXiv:2105.04714*, 2021. Local copy: refs/Guo2021.pdf (retrieved 2026-08-04; screened, see refs/SCAN.md). (cited on p. 45)

[Gen11] Jason Geng. Structured-light 3d surface imaging: a tutorial. *Advances in Optics and Photonics*, 3(2):128–160, 2011. No local copy retrievable from open-access sources (2026-08-04). (cited on p. 9)

[GLW+21] Zheng Ge, Songtao Liu, Feng Wang, Zeming Li, and Jian Sun. YOLOX: Exceeding YOLO series in 2021. *arXiv preprint arXiv:2107.08430*, 2021. Local copy: refs/Ge2021.pdf (retrieved 2026-08-04; screened, see refs/SCAN.md). (cited on p. 45)

[GNH19] Patrick Grother, Mei Ngan, and Kayee Hanaoka. Face recognition vendor test (FRVT) part 2: Identification. Technical Report NISTIR 8271, National Institute of Standards and Technology, 2019. Local copy: refs/Grother2019.pdf (retrieved 2026-08-04; screened, see refs/SCAN.md). (cited on p. 43)

[GV13] Gene H. Golub and Charles F. Van Loan. *Matrix Computations*. Johns Hopkins University Press, fourth edition, 2013. Book; no local copy. (cited on pp. 6 and 28)

[Har97] Richard I. Hartley. In defense of the eight-point algorithm. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 19(6):580–593, 1997. Local copy: refs/Hartley1997.pdf (retrieved 2026-08-04; screened, see refs/SCAN.md). (cited on pp. 17 and 28)

[HDD+92] Hugues Hoppe, Tony DeRose, Tom Duchamp, John McDonald, and Werner Stuetzle. Surface reconstruction from unorganized points. In *Proceedings of SIGGRAPH 92*, pages 71–78, 1992. Local copy: refs/Hoppe1992.pdf (retrieved 2026-08-04; screened, see refs/SCAN.md). (cited on p. 21)

[Hor87] Berthold K. P. Horn. Closed-form solution of absolute orientation using unit quaternions. *Journal of the Optical Society of America A*, 4(4):629–642, 1987. Local copy: refs/Horn1987.pdf (retrieved 2026-08-04; screened, see refs/SCAN.md). (cited on p. 27)

[HRBLM07] Gary B. Huang, Manu Ramesh, Tamara Berg, and Erik Learned-Miller. Labeled faces in the wild: A database for studying face recognition in unconstrained environments. Technical Report 07-49, University of Massachusetts, Amherst, 2007. Local copy: refs/Huang2007.pdf (retrieved 2026-08-04; screened, see refs/SCAN.md). (cited on pp. 43 and 45)

[HS97] Richard I. Hartley and Peter Sturm. Triangulation. *Computer Vision and Image Understanding*, 68(2):146–157, 1997. Local copy: refs/HartleySturm1997.pdf (retrieved 2026-08-04; screened, see refs/SCAN.md). (cited on pp. 18, 38, and 50)

[HWB+13] Armin Hornung, Kai M. Wurm, Maren Bennewitz, Cyrill Stachniss, and Wolfram Burgard. OctoMap: An efficient probabilistic 3d mapping framework based on octrees. *Autonomous Robots*, 34(3):189–206, 2013. Local copy: refs/Hornung2013.pdf (retrieved 2026-08-04; screened, see refs/SCAN.md). (cited on p. 21)

[HZ04] Richard Hartley and Andrew Zisserman. *Multiple View Geometry in Computer Vision*. Cambridge University Press, second edition, 2004. Book; no local copy. (cited on pp. 6, 17, and 18)

[ISM84] Seiji Inokuchi, Kosuke Sato, and Fumio Matsuda. Range imaging system for 3-d object recognition. In *Proceedings of the International Conference on Pattern Recognition*, pages 806–808, 1984. No local copy retrievable from open-access sources (2026-08-04). (cited on p. 9)

[KBH06] Michael Kazhdan, Matthew Bolitho, and Hugues Hoppe. Poisson surface reconstruction. In *Proceedings of the Fourth Eurographics Symposium on Geometry Processing*, pages 61–70, 2006. Local copy: refs/Kazhdan2006.pdf (retrieved 2026-08-04; screened, see refs/SCAN.md). (cited on p. 20)

[KLL+13] Maik Keller, Damien Lefloch, Martin Lambers, Shahram Izadi, Tim Weyrich, and Andreas Kolb. Real-time 3d reconstruction in dynamic scenes using point-based fusion. In *Proceedings of the International Conference on 3D Vision*, pages 1–8, 2013. Local copy: refs/Keller2013.pdf (retrieved 2026-08-04; screened, see refs/SCAN.md). (cited on p. 21)

[KS00] Kiriakos N. Kutulakos and Steven M. Seitz. A theory of shape by space carving. *International Journal of Computer Vision*, 38(3):199–218, 2000. Local copy: refs/KutulakosSeitz2000.pdf (retrieved 2026-08-04; screened, see refs/SCAN.md). (cited on p. 24)

[Lau94] Aldo Laurentini. The visual hull concept for silhouette-based image understanding. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 16(2):150–162, 1994. No local copy retrievable from open-access sources (2026-08-04). (cited on p. 24)

[LC87] William E. Lorensen and Harvey E. Cline. Marching cubes: A high resolution 3d surface construction algorithm. In *Proceedings of SIGGRAPH 87*, pages 163–169, 1987. No local copy retrievable from open-access sources (2026-08-04). (cited on pp. 20, 21, and 22)

[LH81] H. C. Longuet-Higgins. A computer algorithm for reconstructing a scene from two projections. *Nature*, 293:133–135, 1981. No local copy retrievable from open-access sources (2026-08-04). (cited on p. 17)

[Low04] David G. Lowe. Distinctive image features from scale-invariant keypoints. *International Journal of Computer Vision*, 60(2):91–110, 2004. Local copy: refs/Lowe2004.pdf (retrieved 2026-08-04; screened, see refs/SCAN.md). (cited on p. 26)

[MAD+18] Brianna Maze, Jocelyn Adams, James A. Duncan, Nathan Kalka, Tim Miller, Charles Otto, Anil K. Jain, W. Tyler Niggel, Janet Anderson, Jordan Cheney, and Patrick Grother. IARPA janus benchmark-C: Face dataset and protocol. In *Proceedings of the International Conference on Biometrics*, pages 158–165, 2018. Local copy: refs/Maze2018.pdf (retrieved 2026-08-04; screened, see refs/SCAN.md). (cited on pp. 43 and 45)

[NIH+11] Richard A. Newcombe, Shahram Izadi, Otmar Hilliges, David Molyneaux, David Kim, Andrew J. Davison, Pushmeet Kohli, Jamie Shotton, Steve Hodges, and Andrew Fitzgibbon. KinectFusion: Real-time dense surface mapping and tracking. In *Proceedings of the IEEE International Symposium on Mixed and Augmented Reality*, pages 127–136, 2011. Local copy: refs/Newcombe2011.pdf (retrieved 2026-08-04; screened, see refs/SCAN.md). (cited on pp. 20 and 21)

[OK93] Masatoshi Okutomi and Takeo Kanade. A multiple-baseline stereo. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 15(4):353–363, 1993. No local copy retrievable from open-access sources (2026-08-04). (cited on p. 24)

[RF18] Joseph Redmon and Ali Farhadi. YOLOv3: An incremental improvement. *arXiv preprint arXiv:1804.02767*, 2018. Local copy: refs/Redmon2018.pdf (retrieved 2026-08-04; screened, see refs/SCAN.md). (cited on p. 42)

[SCD+06] Steven M. Seitz, Brian Curless, James Diebel, Daniel Scharstein, and Richard Szeliski. A comparison and evaluation of multi-view stereo reconstruction algorithms. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 519–528, 2006. Local copy: refs/Seitz2006.pdf (retrieved 2026-08-04; screened, see refs/SCAN.md). (cited on p. 25)

[SKP15] Florian Schroff, Dmitry Kalenichenko, and James Philbin. FaceNet: A unified embedding for face recognition and clustering. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 815–823, 2015. Local copy: refs/Schroff2015.pdf (retrieved 2026-08-04; screened, see refs/SCAN.md). (cited on p. 42)

[SM99] Peter F. Sturm and Stephen J. Maybank. On plane-based camera calibration: A general algorithm, singularities, applications. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 432–437, 1999. No local copy retrievable from open-access sources (2026-08-04). (cited on pp. 7, 9, and 11)

[SS02] Daniel Scharstein and Richard Szeliski. A taxonomy and evaluation of dense two-frame stereo correspondence algorithms. *International Journal of Computer Vision*, 47(1–3):7–42, 2002. Local copy: refs/ScharsteinSzeliski2002.pdf (retrieved 2026-08-04; screened, see refs/SCAN.md). (cited on p. 19)

[SSS06] Noah Snavely, Steven M. Seitz, and Richard Szeliski. Photo tourism: Exploring photo collections in 3d. *ACM Transactions on Graphics*, 25(3):835–846, 2006. Local copy: refs/Snavely2006.pdf (retrieved 2026-08-04; screened, see refs/SCAN.md). (cited on p. 26)

[SWK07] Ruwen Schnabel, Roland Wahl, and Reinhard Klein. Efficient RANSAC for point-cloud shape detection. *Computer Graphics Forum*, 26(2):214–226, 2007. No local copy retrievable from open-access sources (2026-08-04). (cited on pp. 20 and 22)

[TMHF00] Bill Triggs, Philip F. McLauchlan, Richard I. Hartley, and Andrew W. Fitzgibbon. Bundle adjustment — a modern synthesis. In *Vision Algorithms: Theory and Practice*, volume 1883 of *Lecture Notes in Computer Science*, pages 298–372. Springer, 2000. No local copy retrievable from open-access sources (2026-08-04). (cited on pp. 18, 26, and 49)

[Tsa87] Roger Y. Tsai. A versatile camera calibration technique for high-accuracy 3d machine vision metrology using off-the-shelf TV cameras and lenses. *IEEE Journal of Robotics and Automation*, 3(4):323–344, 1987. No local copy retrievable from open-access sources (2026-08-04). (cited on p. 7)

[Zha00] Zhengyou Zhang. A flexible new technique for camera calibration. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 22(11):1330–1334, 2000. No local copy retrievable from open-access sources (2026-08-04). (cited on pp. 7 and 8)